

rubrapack Manual

Building and signing MSI and MSIX packages - and how the formats work

Version 0.7.0

<https://github.com/rubidus-api/rubrapack>
<https://rubidus-api.github.io/rubrapack/>

The English edition is the original.
MIT License.

Contents

Part I - Tutorial

1. Before you start	2
1.1 What an installer does	2
1.2 What you write, what rubrapack makes	2
1.3 Getting rubrapack	3
1.4 A terminal, for those who have not used one	3
1.5 The folder you work in	3
1.6 Words used in this tutorial	4
2. A first installer: one program	5
2.1 The source	5
2.2 Building	6
2.3 Installing and removing it	6
2.4 What happened inside	6
2.5 Check it without building	7
2.6 The same, with questions instead of a text editor	7
3. Versions and upgrades	8
3.1 One source for every version	8
3.2 What Windows does with two versions	9
3.3 Version numbers	9
3.4 Two identifiers	9
3.5 Check the upgrade before users do	10
3.6 Earlier versions made by another tool	10

Part II - Reference

4. Source files and the rubrapack command	12
4.1 Getting started	13
4.2 Example	16
4.3 The TOML subset	16
4.4 Tables	16
4.5 Command line	27
4.6 Diagnostic codes	31

Part III - File formats

5. Package formats for implementers	34
5.1 How to read the facts	35
5.2 Conventions	35
5.3 The pipeline, in one picture	35
6. Compound File Binary (CFB)	36
6.1 Sectors	36
6.2 Header (first 512 bytes) [spec]	36
6.3 FAT, DIFAT, mini FAT [spec]	37
6.4 Directory [spec]	37
6.5 A simple, deterministic writer	38
6.6 Reading safely	38
7. The MSI database inside the compound file	39
7.1 Streams	39
7.2 The string pool: _StringPool and _StringData	39
7.3 System tables	40
7.4 Table streams	41

7.5 IDT archive files (what MsiDatabaseExport writes)	41
7.6 Reading safely	42
8. Summary information	43
8.1 Layout [spec] [observed]	43
8.2 Properties an installer package sets	43
8.3 The code page trap - keep summary strings ASCII [observed]	44
9. The smallest package that installs, repairs, upgrades and uninstalls	45
9.1 Tables and columns	45
9.2 Property	46
9.3 Directory	46
9.4 Components, features, files	46
9.5 Empty folders	47
9.6 Media and the cabinet	47
9.7 Major upgrade and downgrade refusal	47
9.8 Running an installed program to register and unregister	47
9.9 Registry values	48
9.10 Shortcuts	48
9.11 Removing files and duplicating them	48
9.12 Environment variables	49
9.13 INI files	49
9.14 Searches and launch conditions	49
9.15 Services, fonts, permissions	50
9.16 Cabinets, administrative images, advertisement	50
9.17 Per-user and dual packages	50
9.18 Files in use	50
9.19 Sequences	50
9.20 Dialogs	51
9.21 What this recipe does not cover yet	53
10. Program files: machine type and version	54
10.1 Finding the headers [spec]	54
10.2 The version resource [spec]	54
10.3 What goes into the package [observed]	54
10.4 Reading safely	55
11. Cabinets, MSZIP and deflate	56
11.1 Cabinet layout [spec: MS-CAB]	56
11.2 MSZIP [spec: MS-MCI]	56
11.3 A deflate encoder in brief [spec: RFC 1951]	57
12. Deterministic identities and reproducible packages	58
12.1 Name-based GUIDs (UUID version 8) [spec: RFC 9562]	58
12.2 Keys	58
12.3 Reproducible packages	59
13. Authenticode signatures	60
13.1 Where the signature lives in a PE file	60
13.2 The PE digest	60
13.3 The CMS SignedData	61
13.4 MSI packages	61
13.5 Timestamps	62
13.6 ECDSA	62
13.7 What Windows reports	62
14. Checking your output against Windows	63
14.1 The database opens and decodes	63

14.2 The package installs	63
14.3 Checks before writing	64
14.4 Microsoft's ICE rules [observed]	64
14.5 Things that look fine but are not	65
15. MSIX packages	66
15.1 The ZIP archive	66
15.2 The block map (AppxBlockMap.xml)	66
15.3 [Content_Types].xml	67
15.4 The manifest (AppxManifest.xml)	67
15.5 The virtual registry (Registry.dat, User.dat)	67
15.6 The virtual file system (VFS\...)	68
15.7 Extensions	68
15.8 Bundles (.msixbundle)	69
15.9 Signatures (AppxSignature.p7x)	70
15.10 Installing an unsigned package	70
16. Registry hive files (REGF)	71
16.1 Layout	71
16.2 The base block	71
16.3 Keys (nk)	72
16.4 Subkey lists (lh, ri)	72
16.5 Values (vk) and value lists	72
16.6 The security cell (sk)	72
16.7 Reproducible hives	72

Part I

Tutorial

1 Before you start

In this chapter

1.1	What an installer does	2
1.2	What you write, what rubrapack makes	2
1.3	Getting rubrapack	3
1.4	A terminal, for those who have not used one	3
1.5	The folder you work in	3
1.6	Words used in this tutorial	4

This part of the manual is a tutorial. It starts with the smallest installer there is - one program copied into Program Files - and adds one thing per chapter until every feature of rubrapack has been used once. Each chapter builds on the one before, so it is best read in order. Nothing is assumed beyond using a computer: where a chapter needs an idea from outside (a GUID, a hash, a registry key), it explains it on the spot or points to the Background knowledge part.

1.1 What an installer does

A program on your own computer is just files in a folder. To give it to someone else, you could send them the folder - but then they have to choose where to put it, make their own Start menu entry, and later find and delete everything by hand. An *installer* does this for them:

- it copies the files to the right place (usually C:\Program Files\<your program>);
- it creates shortcuts, registry values, file type links and whatever else the program needs;
- it registers the program in Settings > Apps > Installed apps, so it can be removed there;
- when you ship version 2, it replaces version 1 cleanly;
- if something goes wrong halfway, it puts everything back as it was.

Windows has a built-in engine for this, **Windows Installer**. You do not write a program that installs; you write a *package*, a file that describes what to install, and Windows Installer carries it out. Two package formats exist:

Format	File	Installed by	Typical use
Windows Installer	.msi	Windows Installer (msiexec), since Windows 2000	desktop programs of every kind; companies deploy MSI through their management tools
MSIX	.msix, .msixbundle	the App Installer of Windows 10 and 11	modern packaged apps, the Microsoft Store

rubrapack makes both, from the same description. Most of this tutorial is about MSI; chapter 17 turns the same example into an MSIX.

1.2 What you write, what rubrapack makes

You write one short text file, the *source*, in [TOML](#) - a simple key = value format. rubrapack reads it and writes the package:

```
hello.toml (you write this)
|
| rubrapack build hello.toml -o hello.msi
```

```

      v
hello.msi  (you give this to users)
      |
      |    double-click, or: msixec /i hello.msi
      v
C:\Program Files\Hello\hello.exe  (Windows Installer puts it there)

```

An MSI file is itself a small database with dozens of tables, identifiers and a compressed archive inside - Part IV of this book shows it byte by byte. You never have to write those tables: rubrapack works them out from the source.

1.3 Getting rubrapack

rubrapack is one program file with nothing else to install. From the releases page, <https://github.com/rubidus-api/rubrapack/releases>, download:

- on Windows: rubrapack-<version>-windows-x64.exe. Rename it to rubrapack.exe.
- on Linux: rubrapack-<version>-linux-x86_64. Rename it to rubrapack and make it executable: `chmod +x rubrapack`.

Put it in a folder that is on your PATH (so the command works from anywhere), or keep it in the folder you work in and type `.\rubrapack.exe` (Windows) or `./rubrapack` (Linux) instead of rubrapack. This tutorial writes plain rubrapack.

You can build packages on Linux, but you need Windows to install and try them. A virtual machine with Windows is enough.

1.4 A terminal, for those who have not used one

rubrapack has no window: you type commands. On Windows, press the Windows key, type terminal (or cmd), and press Enter. A window with a prompt opens, for example:

```
C:\Users\you>
```

The part before > is the *current folder*. Commands work on files in that folder unless you give a path. Three commands are all you need:

Command	What it does
<code>cd C:\work\hello</code>	go into the folder C:\work\hello
<code>dir</code> (Linux: <code>ls</code>)	list the files in the current folder
<code>mkdir dist</code>	make a folder named dist here

Try it: `rubrapack version` prints the version, and `rubrapack help` lists the commands.

```
C:\work\hello> rubrapack version
rubrapack 0.7.0 (proven_c_lib-v0.1.1)
```

1.5 The folder you work in

Every chapter uses the same layout. Make a folder for your project, and inside it a folder dist ("distribution") holding exactly the files you ship:

```

C:\work\hello\
  dist\
    hello.exe      the program - any program of your own will do
    hello.toml     the source (the next chapter writes it)

```

If you have no program at hand, any .exe works for learning; the installer does not run it.

1.6 Words used in this tutorial

Word	Meaning
package	the .msi (or .msix) file you give to users
source	the .toml file you write; rubrapack builds the package from it
install folder	where the files go on the user's computer, e.g. C:\Program Files\Hello
product	your program as Windows sees it: a name, a version, a manufacturer
upgrade code	a fixed identifier that ties every version of your product together (chapter 2)
feature	a part of the product the user may choose to install or not (chapter 8)
component	Windows Installer's unit of installation; rubrapack makes these for you

2 A first installer: one program

In this chapter

2.1	The source	5
2.2	Building	6
2.3	Installing and removing it	6
2.4	What happened inside	6
2.5	Check it without building	7
2.6	The same, with questions instead of a text editor	7

Goal: an installer that copies `hello.exe` into `C:\Program Files\Hello`, shows up in Installed apps, and removes itself cleanly.

2.1 The source

In the folder `C:\work\hello` (see the previous chapter), create a text file `hello.toml` with any text editor - Notepad will do - and write:

```
# tutorial 02: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "1.0.0"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"
```

Line by line:

- A line starting with `#` is a comment; `rubrapack` ignores it.
- `[package]` starts a *table*: the lines after it, up to the next `[ ... ]`, belong to it. The package table says what the product is.
 - `name` is what users see: in Installed apps, in the title of the installer's windows.
 - `manufacturer` is you or your company; Installed apps shows it under the name.
 - `version` has three numbers (or four): major, minor, build. Chapter 3 is about changing it.
 - `arch` is the processor the program is built for: `x64` (almost every PC today), `x86` (32-bit programs) or `arm64`. `rubrapack` checks that `hello.exe` really is an `x64` program.
 - `upgrade-code` is a GUID - a 128-bit number written as 32 hexadecimal digits in braces - that is unique to your product. Make your own with `rubrapack guid` (below), write it here once, and **never change it**: every later version must carry the same one, or Windows will treat it as a different product and install it next to the old one.
- `[dir.INSTALLDIR]` declares a folder with the ID `INSTALLDIR`. An ID is a name you choose (letters, digits and `_`) to refer to a thing from other tables. `path` says where the folder is: `ProgramFiles/`

Hello means C:\Program Files\Hello on an English or Korean Windows alike - the known folder ProgramFiles is resolved on the user's computer.

- [file.Hello] installs one file, with the ID Hello. dir names the folder it goes into, by its ID; source is where the file is now, relative to hello.toml, written with /.

Get your own upgrade code - a new one every time you run it:

```
C:\work\hello> rubrapack guid
{F1B1ED21-3061-4A65-8D77-FBB5ACCB1642}
```

Copy it over the one in the example. (Using the example's code would make your product and every other reader's "the same product" to Windows.)

2.2 Building

```
C:\work\hello> rubrapack build hello.toml -o hello.msi
```

Nothing is printed when all is well, and hello.msi (about 28 KB) appears next to the source. If something is wrong, rubrapack says where and why and writes nothing, for example after a typo in source:

```
hello.toml:14:1: error[RP1507]: source file 'dist/hello.ex' not found
```

The numbers are the line and the column; the code in brackets (RP1507: RP15xx is "the files to install") is explained under Diagnostic codes in the Reference part. Fix the line and build again.

2.3 Installing and removing it

Copy hello.msi to a Windows computer and double-click it. Because it installs into Program Files, which belongs to all users, Windows asks for administrator permission (the User Account Control prompt). Then a small window with a progress bar appears for a moment - this package has no dialogs of its own yet (chapter 6 adds them). Afterwards:

- C:\Program Files\Hello\hello.exe exists;
- Settings > Apps > Installed apps lists **Hello**, version 1.0.0, by Example Software.

Remove it from Installed apps (the . . . menu, Uninstall), or from a terminal:

```
msiexec /x hello.msi
```

The file and the folder are gone again. The same package can also be installed without any window, which is how companies deploy software (/qn means "quiet, no user interface"; run the terminal as administrator):

```
msiexec /i hello.msi /qn
msiexec /x hello.msi /qn
```

When something goes wrong on Windows, a log tells why: msiexec /i hello.msi /l*v install.log writes everything Windows Installer did into install.log.

2.4 What happened inside

rubrapack inspect shows what the package holds without installing it. --files lists the files and where they go:

```
C:\work\hello> rubrapack inspect hello.msi --files
[ProgramFiles64Folder]\Hello\hello.exe 17920 Hello C_185f8db32271fe25f561 1.2.3.4 1033
```

ProgramFiles64Folder is Windows Installer's name for 64-bit Program Files; 17920 is the size in bytes; 1.2.3.4 and 1033 (US English) are the version and language written inside `hello.exe` (Windows compares them when it replaces a file). An MSI is a database, and `inspect` can print any of its tables; the File table has one row for our one file:

```
C:\work\hello> rubrapack inspect hello.msi File
File Component_ FileName FileSize Version Language Attributes Sequence
s72 s72 l255 i4 S72 S20 I2 i4
File File
Hello C_185f8db32271fe25f561 hello.exe 17920 1.2.3.4 1033 512 1
```

The first line names the columns, the second gives their types (`s72` is a string of up to 72 characters, `i4` a 4-byte integer, a capital letter means the column may be empty), the third names the table and its key column. `Hello` is your file ID; `C_185f...` is a *component* that `rubrapack` made for it; 512 means "vital" (the installation fails if this file cannot be written). The Property table holds the product's identity:

```
C:\work\hello> rubrapack inspect hello.msi Property
...
ProductCode {98BFA9A2-2741-81F9-A1CA-2C6626AE1304}
ProductName Hello
ProductVersion 1.0.0
UpgradeCode {3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}
...
```

The `ProductCode` is another GUID, which identifies *this version*: `rubrapack` derives it from the upgrade code, the architecture and the version, so it changes by itself when the version does. You do not write it. Part IV explains how these tables are stored in the file.

2.5 Check it without building

`rubrapack lint hello.toml` runs every check the build runs and writes nothing - handy after an edit. `rubrapack lint hello.msi` checks a finished package, even one made by another tool:

```
C:\work\hello> rubrapack lint hello.msi
hello.msi: 0 errors, 0 warnings
```

2.6 The same, with questions instead of a text editor

`rubrapack new hello.toml` asks for the name, the version, the folder with the files and so on, and writes a source like the one above (it also makes up the upgrade code). Chapter 19 describes it and its companion `rubrapack edit`. Writing the source by hand once, as here, is the best way to learn what each line does.

3 Versions and upgrades

In this chapter

3.1	One source for every version	8
3.2	What Windows does with two versions	9
3.3	Version numbers	9
3.4	Two identifiers	9
3.5	Check the upgrade before users do	10
3.6	Earlier versions made by another tool	10

Goal: ship version 1.1.0 of Hello so that installing it replaces 1.0.0, and installing 1.0.0 again afterwards is refused.

3.1 One source for every version

Editing version by hand before each release works, but it is easy to forget. Instead, let the version be a *variable* with a default, and give the real number when you build:

```
# tutorial 03: hello.toml
[package]
name = "Hello"
manufacturer = "Example Software"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D}"
downgrade-message = "A newer Hello is already installed. Remove it first if you want this version."

[define]
VERSION = "1.0.0"

[dir.INSTALLDIR]
path = "ProgramFiles/Hello"

[file.Hello]
dir = "INSTALLDIR"
source = "dist/hello.exe"
```

- [define] holds variables. `${VERSION}` anywhere in a string is replaced by the variable's value; here the default is `1.0.0`.
- `-D NAME=VALUE` on the command line overrides a variable for one build:

```
C:\work\hello> rubrapack build hello.toml -o hello-1.0.0.msi
C:\work\hello> rubrapack build hello.toml -o hello-1.1.0.msi -D VERSION=1.1.0
```

- `downgrade-message` is the message a user sees when they try to install an older (or the same) version over a newer one. Without it, rubrapack uses "The same or a newer version of [ProductName] is already installed." ([ProductName] is replaced by the name).

3.2 What Windows does with two versions

Install `hello-1.0.0.msi`, then `hello-1.1.0.msi`. Windows Installer sees that the new package has the same upgrade code and a higher version, removes 1.0.0 and installs 1.1.0 - one entry in Installed apps, now showing 1.1.0. This is called a *major upgrade*, and it is the only kind of upgrade rubrapack builds: every new version is a complete package, and nobody needs the old one to install the new one.

Now try to install `hello-1.0.0.msi` again: Windows shows the downgrade message and changes nothing. Running `hello-1.1.0.msi` a second time is not an error either: it has the same product code as the installed version (see below), so Windows recognises the product as installed and does not install it twice. To repair an installed version - put back a deleted file, for example - run `msiexec /fa hello-1.1.0.msi`.

A package rebuilt from changed files *without* raising the version has the same product code as the installed one but is not the same package, and Windows refuses it with its own message: "Another version of this product is already installed" (error 1638). So raise the version whenever the files change - only a higher version replaces what is installed.

How rubrapack arranges this is visible in the Upgrade table:

```
C:\work\hello> rubrapack inspect hello-1.1.0.msi Upgrade
UpgradeCode VersionMin VersionMax Language Attributes Remove ActionProperty
s38 S20 S20 S255 i4 S255 s72
Upgrade UpgradeCode VersionMin VersionMax Language Attributes
{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D} 1.1.0 1 RP_OLDER_FOUND
{3F2A6C1D-8B4E-4F7A-9C2D-5E6F7A8B9C0D} 1.1.0 258 RP_NEWER_FOUND
```

The first row finds installed versions below 1.1.0 (to remove them); the second finds 1.1.0 or higher, which stops the installation with the downgrade message. The attribute numbers are bit flags - 258 is 256 + 2, "the minimum is included" plus "only detect, do not remove". Part III explains bit flags, Part IV this table.

3.3 Version numbers

- Three or four numbers: `major.minor.build` or `major.minor.build.revision`. Windows Installer allows at most 255.255.65535 for the first three.
- **Windows compares only the first three.** `1.0.0.1` and `1.0.0.2` are the same version to it: the second would be refused as "the same". Raise the third number (or a higher one) for every release.
- A wrong version stops the build:

```
hello.toml:5:1: error[RP1308]: version '1.0' must be a.b.c or a.b.c.d with a, b <= 255 and c, d <= 65535
```

3.4 Two identifiers

	Upgrade code	Product code
identifies	the product across all its versions	one version
you write it	once, in upgrade-code; never change it	no: rubrapack derives it
changes	never	whenever the version (first three numbers) changes

```
1.0.0: ProductCode {98BFA9A2-2741-81F9-A1CA-2C6626AE1304}
1.1.0: ProductCode {9CC17977-2023-8B8F-8BB0-2FA82C5B248F}
```

The derived product code is the same every time you build the same version, on any computer. product-code in [package] sets it by hand - only needed to continue a product whose earlier versions came from another tool with a product code of their own.

3.5 Check the upgrade before users do

Some mistakes do not stop a build but break the upgrade on users' computers: a changed upgrade code, a version that is not higher, a file that moved in a way Windows cannot follow. Compare the new package with the one before it:

```
C:\work\hello> rubrapack lint hello-1.1.0.msi --previous hello-1.0.0.msi
hello-1.1.0.msi: 0 errors, 0 warnings (against hello-1.0.0.msi)
```

and the same package against itself, to see what a mistake looks like:

```
C:\work\hello> rubrapack lint hello-1.0.0.msi --previous hello-1.0.0.msi
hello-1.0.0.msi: error[RP2302]: ProductVersion 1.0.0 is not higher than the previous 1.0.0 in
its first three fields: Windows Installer does not upgrade
hello-1.0.0.msi: warning[RP2303]: the ProductCode is the previous version's: that is a minor
upgrade (REINSTALLMODE), not the major upgrade rubrapack builds
hello-1.0.0.msi: 1 error, 1 warning (against hello-1.0.0.msi)
```

Keep every released .msi (for example in a releases folder) so the next one can be checked against it.

3.6 Earlier versions made by another tool

If your product was installed before with packages from another tool, rubrapack's upgrade replaces them as long as they have the same upgrade code. Two keys help when that goes badly:

- `refuse-upgrade-below = "0.60.0"`: versions below this one are not upgraded but refused, with a message that tells the user how to remove them first - for old packages known to break when removed as part of an upgrade.
- `refuse-upgrade-message = "..."`: that message's first part (the removal command is added).

Part II

Reference

4 Source files and the rubrapack command

In this chapter

4.1	Getting started	13
	A first package	13
	A license page and optional parts	14
	Trying it on Windows	15
	When something is wrong	15
	Automating it	15
	With an AI assistant	16
4.2	Example	16
4.3	The TOML subset	16
4.4	Tables	16
	MSIX packages	18
	Registering with an installed program: [action.ID]	19
	Registry values: [registry.ID]	19
	Shortcuts: [shortcut.ID]	19
	File types and links: [assoc.ID], [protocol.ID]	20
	MSIX only: [msix-extension.ID]	20
	Removing and copying: [remove.ID], [copy.ID]	20
	Environment variables: [env.ID]	20
	INI files: [ini.ID]	21
	Searching and requiring: [search.ID], [require.ID]	21
	Services, fonts and permissions	21
	Per-user and dual packages: scope	21
	Cabinets	22
	Installed apps entry and properties	22
	Installing from the command line	22
	Installing without a window	22
	Dialogs: ui	23
	Several languages	23
	Icon, the finished page, and the scope page	24
	Your own dialog pages: [dialog.ID], [dialog-control.ID]	24
	Architectures and upgrade families	25
	Versions that must be removed first	25
	Wildcards: [files.ID]	25
	Empty folders: [folder.ID]	26
	Features	26
	Conditions: when	26
	Variables	26
	Program files	26
	Leaving a file at removal: keep	27
	Guarding the install folder	27
	Paths	27
4.5	Command line	27
4.6	Diagnostic codes	31

A source file describes a package. It is a strict subset of [TOML 1.0](#) and is named `*.toml`, so editors, GitHub and AI assistants treat it as TOML (the older `*.rpk` name is read the same way). Every source is valid TOML, but rubrapack refuses TOML features outside the subset instead of ignoring them.

4.1 Getting started

rubrapack is a single program file. Download `rubrapack-<version>-windows-x64.exe` (rename it `rubrapack.exe` if you like) or `rubrapack-<version>-linux-x86_64` (`chmod +x` it) from the releases page and run it where it lies, or put it on the PATH. There is nothing else to install: no runtime, no SDK, no libraries. The same program builds the same packages on Windows and on Linux.

A first package

Put what you ship in a folder `dist/`: the program, and whatever it needs, in `dist/files/` (subfolders are kept). Then let rubrapack write a starting source:

```
rubrapack new app.toml      # asks, then writes app.toml and checks it
```

It asks for the product name, the version, the folder with the files, the main program (its architecture is read from the file), the folder under Program Files, who it installs for, which sub folders are optional parts, the dialogs, a license (a `LICENSE.txt`, `.md` or `.rtf` next to it is offered), Korean dialogs, and the shortcuts. Enter takes the value in brackets. At the end it prints the same answers as one command (`rubrapack new app.toml --dist dist --ui features ...`), which a script or an AI assistant can run without questions. `rubrapack new` without a name asks for the file name too, and `rubrapack new app` (no extension) writes a fixed starter instead.

Either way, `app.toml` is plain text to read and change - in any text editor, or with `rubrapack edit app.toml`, a menu of the same questions with the current values as defaults (version, install folder, dialogs and license, optional parts, shortcuts, and matching the file list to the program folder after it changed). It changes only those values and keeps every other line, comment and table as it was. For scripts: `rubrapack edit app.toml --set define.VERSION=1.1.0 --sync`. This one installs the program and its files into Program Files\My App, puts it in the Start menu, and shows a dialog that lets the user change the folder:

```
[package]
name = "My App"
manufacturer = "My Company"           # shown in Settings > Installed apps
version = "${VERSION}"
arch = "x64"                           # x64, arm64 or x86
upgrade-code = "{E8C1815C-CCD7-4F3F-B914-92A4E3F3A317}" # yours from `new`; keep it forever
ui = "installdir"

[define]
VERSION = "1.0.0"

[dir.INSTALLDIR]
path = "ProgramFiles/My App"

[file.App]
dir = "INSTALLDIR"
source = "dist/app.exe"

[files.Rest]
dir = "INSTALLDIR"
glob = "dist/files/**"

[shortcut.StartMenu]
dir = "Programs"
```

```
name = "My App"
target = "file:App"
```

```
rubrapack build app.toml -o app-1.0.0.msi
rubrapack build app.toml -o app-1.0.1.msi -D VERSION=1.0.1    # the next version
```

That is a complete installer. It appears in Installed apps, repairs itself, and removes everything it installed. A higher version replaces the one installed; the same or an older one is refused with a message. The component GUIDs, file keys, cabinet and tables are derived from the source, so nothing but the upgrade code needs to be remembered from one version to the next.

A license page and optional parts

Two things most installers want: the user accepts a license before installing, and some parts are optional. `license` adds the license page (Next stays off until "I accept" is ticked), and `ui = "features"` adds a tree in which the user picks the features to install. Here the program is always installed and the samples are offered but not selected:

```
[package]
name = "My App"
manufacturer = "My Company"
version = "${VERSION}"
arch = "x64"
upgrade-code = "{E8C1815C-CCD7-4F3F-B914-92A4E3F3A317}"
ui = "features"                # welcome, license, folder, feature tree, ready
license = "LICENSE.txt"        # Next stays off until "I accept" is ticked

[define]
VERSION = "1.0.0"

[feature.Main]
title = "My App"
description = "The program itself."
required = true                # always installed: the tree does not offer to leave it
out

[feature.Samples]
title = "Samples"
description = "Example documents to try the program with."
level = 2                      # offered in the tree, not selected by default

[dir.INSTALLDIR]
path = "ProgramFiles/My App"
feature = "Main"

[dir.SamplesDir]
path = "INSTALLDIR/samples"
feature = "Samples"

[file.App]
dir = "INSTALLDIR"
source = "dist/app.exe"

[files.Rest]
dir = "INSTALLDIR"
glob = "dist/files/**"

[files.Samples]
dir = "SamplesDir"
```

```
glob = "dist/samples/**"

[shortcut.StartMenu]
dir = "Programs"
name = "My App"
target = "file:App"
```

Put the license text in LICENSE.txt (.txt, .md or .rtf) and the samples in dist/samples/. A feature with level = 2 is not installed unless the user ticks it; required keeps the tree from offering to leave a feature out. Later the user changes the choice from Installed apps (Change), and the command line does the same without dialogs: `msiexec /i app.msi /qn ADDLOCAL=Samples` adds the samples, `REMOVE=Samples` takes them out (required binds only the tree, not the command line). Features and Conditions have the rest.

Trying it on Windows

```
:: install, with the dialogs
msiexec /i app-1.0.0.msi
:: upgrade, silently (as administrator)
msiexec /i app-1.0.1.msi /qn
:: remove
msiexec /x app-1.0.1.msi
:: a full log when something goes wrong
msiexec /i app-1.0.0.msi /l*v install.log
```

Installing from the command line lists the properties that choose features, the folder and the language without dialogs.

When something is wrong

`rubrapack lint app.toml` runs every check a build runs and writes nothing. Every problem names its place and a code, often with the fix:

```
app.toml:18:1: error[RP1201]: unknown key 'glb' in [files.Rest] (did you mean 'glob'?)
app.toml:25:1: error[RP1301]: ID 'App' is already used (line 17); IDs must differ across all
tables
```

To see what a package holds: `rubrapack inspect app.msi` File prints a table as text, `rubrapack extract app.msi -d out` unpacks the files the way they install, and `rubrapack lint app-1.0.1.msi --previous app-1.0.0.msi` checks that the new version upgrades the old one cleanly. Exit codes are fixed: 0 success, 1 source error, 2 usage, 3 input/output, 4 signing, 5 lint, 6 network.

Automating it

A build is one command with no state outside the source, so it fits any script or CI job. With `--reproducible` the same source gives the same bytes on every machine. On a Linux runner:

```
V=0.7.0 # the release to use
curl -sLo rubrapack "https://github.com/rubidus-api/rubrapack/releases/download/v$V/rubrapack-$V-linux-x86_64"
chmod +x rubrapack
./rubrapack lint app.toml
./rubrapack build app.toml -o "app-$VERSION.msi" -D VERSION="$VERSION" --reproducible \
  --key signer.pfx --pass-env SIGN_PASS --timestamp http://timestamp.digicert.com
```

Signing is optional; leave out `--key` and what follows for an unsigned package. See Command line (sign, `--pkcs11`, `--key-store`) for keys on a token or in the Windows certificate store.

With an AI assistant

The source is short plain text and every error says where and why, so an AI coding assistant can do the whole job: give it this manual (or the book, <https://rubidus-api.github.io/rubrapack/>) and say what to install. It can write `app.toml`, run `rubrapack lint app.toml` until it is clean, build, check the result with `inspect` and `extract`, and turn the steps into a release script or a CI job like the one above. Two things stay yours: keep the upgrade code of the first version, and install the package on Windows once before you ship it.

4.2 Example

```
[define]
VERSION = "1.4.0"

[package]
name = "Example App"
manufacturer = "Example"
version = "${VERSION}"
arch = "x64" # x64, arm64 or x86 - no default
upgrade-code = "{0B9A6C1E-3D2F-4A5B-8C7D-6E5F4A3B2C1D}" # generate your own once, keep it forever
language = "en-US" # or "ko-KR"

[dir.INSTALLDIR]
path = "ProgramFiles/Example App"

[dir.Docs]
path = "INSTALLDIR/docs"

[file.MainExe]
dir = "INSTALLDIR"
source = "dist/app.txt"

[file.Guide]
dir = "Docs"
source = "dist/guide.txt"
name = "User guide.txt"
```

```
rubrapack build example.toml -o example.msi -D VERSION=1.4.1
rubrapack inspect example.msi File
```

4.3 The TOML subset

Accepted: tables `[kind]` and `[kind.ID]`, bare keys (A-Z a-z 0-9 _ -), basic strings "...", with TOML escapes, literal strings '...' (no escapes - use them for backslashes and quotes: 'SOFTWARE\Example'), decimal and 0x integers, true/false, one-type arrays, # comments. UTF-8 with or without a BOM, or UTF-16LE with a BOM; LF or CRLF.

Refused with an error: multi-line strings, inline tables, arrays of tables, dotted and quoted keys, table names with more than two parts, floats, dates, _ in numbers, octal/binary numbers, empty or mixed arrays, keys before the first table. Table and key order never matters.

4.4 Tables

Table	Keys (required in bold)
[package]	name , manufacturer , version (a.b.c or a.b.c.d), arch , upgrade-code , upgrade-code-x64 / -arm64 / -x86, product-code, summary-name (ASCII), language, scope (machine, user, dual), ui (none, basic, minimal, installdir, features), license (.txt, .md, .rtf), reboot

Table	Keys (required in bold)
	(suppress/allow), downgrade-message, compress (none, mszip, mszip:0..mszip:9; default mszip:6), cab (embed or external), cab-max-size (MiB), refuse-upgrade-below, refuse-upgrade-message
[define]	variables: NAME = "value"
[feature.ID]	title , description, level (1-32767), hidden, parent, required, follow-parent, when
[dir.ID]	path = Base/relative/path, feature, guard (true: see Guarding the install folder)
[file.ID]	dir , source , name, vital (default true), any-arch, feature, component-guid, keep, when
[files.ID]	dir , glob , vital, any-arch, feature, keep, when
[folder.ID]	dir , name , keep, feature
[arp]	no-modify, no-repair, help (URL), about (URL), icon (.ico) - how the product shows in Installed apps
[property.ID]	value , secure, hidden - an upper-case public property
[action.ID]	run (file:ID of an .exe in this package), do , undo , check
[registry.ID]	root (HKLM, HKCU, HKCR, HKMU), key , name, value, type, remove, keep, view, with, feature, when
[remove.ID]	dir , name (* and ?; omitted = the folder itself), on (install, uninstall, both), feature
[ini.ID]	dir , file , section , key , value, mode (set, add, remove), feature, when
[require.ID]	condition , message
[search.ID]	property (or a dir ID), kind (registry: root, key, name, view; file: path, file, min-version; dir: path; component: component-guid)
[service.ID]	file (file:ID of an .exe), name , display-name, description, start (auto, demand, disabled), account (LocalSystem, LocalService, NetworkService), args, start-on-install
[assoc.ID]	extension (.ext, lower case), prog-id , target (file:ID of an .exe), description, icon (file:ID), args (default "%1")
[protocol.ID]	name (the scheme, lower case), target (file:ID of an .exe), description, args (default "%1")
[font.ID]	file (file:ID of a file in a dir with path = "Fonts"), title
[permission.ID]	target (dir:ID, file:ID, registry:ID), sddl
[env.ID]	name , value , mode (set, append, prepend), keep, feature, when
[copy.ID]	source (file:ID), dir , name (default: the source's name)
[ui]	install-dir (a dir ID; default INSTALLDIR), banner (.bmp), launch (file:ID), launch-args, launch-checked, languages (added to English, e.g. ["ko"]), license-xx, name-xx, font-xx, langid-xx - see Several languages
[ui-text.ID]	text or text-xx - replaces one built-in dialog text
[dialog.ID]	after (a built-in page or another [dialog.*]), title, description, title-xx, description-xx
[dialog-control.ID]	dialog , type (text, checkbox, edit, radio, combo), x , y , width , height , text, property, values, labels, text-xx, labels-xx
[shortcut.ID]	dir (a dir ID, or Programs, Desktop, StartMenu, Startup), name , target (file:ID), args, description, working-dir (a dir ID), icon (.ico), when
[msix]	identity-name , publisher , publisher-display-name, min-version - see MSIX packages
[msix-app.ID]	executable (a [file.*] ID), display-name, description, logo-150, logo-44, store-logo
[msix-extension.ID]	kind (alias: alias ; startup-task: task-id, display-name, enabled), app (an [msix-app.*] ID; default the first) - MSIX only

Base in a dir path is another dir ID or one of: ProgramFiles (64-bit for x64/arm64, 32-bit for x86), ProgramFiles32, CommonFiles, AppData, LocalAppData, CommonAppData, StartMenu, Programs, Desktop, Startup, Windows, System, Fonts, Temp. A path may also be a known folder alone (path = "Fonts") for files that go into that folder itself.

IDs are [A-Za-z_] [A-Za-z0-9_]* (at most 72 characters, 38 for features) and must differ across dirs, files and features.

MSIX packages

The same source builds an MSIX package when the output ends in .msix: one desktop application that runs with full trust, for one architecture. Two more tables say what only MSIX needs:

```
[msix]
identity-name = "Example.App"           # 3-50 characters: A-Z a-z 0-9 . -
publisher = "C=KR, O=Example, CN=Example" # the signing certificate's subject, last part
first
publisher-display-name = "Example"       # default: [package] manufacturer
min-version = "10.0.17763.0"             # the oldest Windows it installs on (this is the
default)

[msix-app.Main]
executable = "MainExe"                   # the [file.*] that starts the app
display-name = "Example App"              # default: [package] name
description = "An example"                # default: the display name
logo-150 = "assets/Square150x150.png"     # PNG, 150x150
logo-44 = "assets/Square44x44.png"        # PNG, 44x44
store-logo = "assets/StoreLogo.png"       # PNG, 50x50
```

- The version is [package] version with four parts (1.2.3 becomes 1.2.3.0), the architecture [package] arch, the language [package] language.
- The package's own folder is that of the first [msix-app.*]'s executable - the dir anchored in a known location, such as ProgramFiles/Example App. Files in other known locations go into the package's virtual file system, where the app sees them at their usual place: Program Files (VFS\ProgramFilesX64, or X86 for an x86 package and for ProgramFiles32), CommonFiles, System, Windows and CommonAppData (ProgramData). There is none for the user's AppData or LocalAppData, nor for Temp: files there are an error (RP1609). A font in Fonts goes in through its [font.*]; the Start menu, Programs and Desktop folders take shortcuts ([shortcut.*]), and Startup a startup task ([msix-extension.*]).
- Several [msix-app.*] tables make several entries of one package (at most 100); the first one gives the package its logo.
- [registry.*] values go into the package's virtual registry, which the app sees merged into the real one while the machine's registry stays untouched: HKLM (and HKMU) under Software into Registry.dat (with view = "32" in the 32-bit view), HKCU under Software into User.dat. What an MSIX cannot hold is an error (RP1612): HKCR (file types and protocols are [assoc.*] and [protocol.*]), keys outside Software, remove and keep, and values with a part Windows Installer fills in at install time ([INSTALLDIR], [#File], ...; the escapes [\[] and [\]] are fine).
- Give the three logos or none: without them the package gets plain one-colour logos. A logo must have the exact size (RP1608).
- What an MSIX cannot do is an error, not something left out quietly (RP1605): custom actions, services, environment variables, INI files, permissions, launch conditions and searches, files removed or copied at install, empty folders. Add msi-only = true to such a table (or to a [file.*]/[files.*]) and the MSI keeps it while the MSIX is built without it. Features, properties, dialogs and [arp] concern the Windows Installer only and are not used for an MSIX.

- `--unsigned-test` adds the attribute Windows needs to install an unsigned package for testing (Add-AppxPackage -AllowUnsigned, as administrator when it contains a program); such a package is not for distribution and its identity differs from the signed one. `--key` signs the package (or the bundle and its packages) instead; see `sign` below.
- Files are compressed (`--msix-compress` store turns it off); pictures, archives and other already compressed files are stored. An MSIX holds no time: the same source gives the same bytes on Linux and Windows.
- An output ending in `.msixbundle` is a bundle: the source built once for each architecture of `--arch` (a list, `--arch x64,x86,arm64`; without it the source's own), each package named `<identity-name>_<version>_<arch>.msix` inside. Windows installs the package for its own architecture from it (an x64 machine takes x64 before x86). `$(ARCH)` gives each build its own programs: `source = "bin/$(ARCH)/app.exe"`. The bundle's version is the packages' version.

Registering with an installed program: [action.ID]

```
[action.Tip]
run = "file:MainExe"      # an .exe this package installs
do = "--register"         # run after the files are installed, and again on repair
undo = "--unregister"     # run on removal, before the files are removed
```

rubrapack turns the pair into deferred, elevated actions with their rollback twins, so removal and upgrade are all-or-nothing: if anything fails later (or do/undo itself exits non-zero), the files come back and the other command restores the registration. Both commands must be safe to run twice and must finish without asking anything - they run without a window, and nothing waits for a user. check names an optional command that exits 0 when the registration is in place.

Registry values: [registry.ID]

```
[registry.InstallDir]
root = "HKLM"
key = 'SOFTWARE\Example'      # literal strings keep the backslashes
name = "InstallDir"          # omit it for the key's default value
value = "[INSTALLDIR]"       # an MSI formatted string: [PROPERTY], [#FileID], [\[] for
"["
```

type is string (default), expand, dword (an integer, 0 to 0xFFFFFFFF), qword (an integer, or "0x" and up to 16 hex digits), binary (hex digits) or multi (an array of strings). Windows Installer cannot write REG_QWORD itself; rubrapack adds its small helper DLL to the package for it, which also restores the previous value if the installation fails. Each value is its own component, removed at uninstall (keep = true leaves it); with = "file:ID" puts it in that file's component instead. In a 64-bit package values go to the 64-bit registry view; view = "32" writes to the 32-bit view. remove = true (without value) deletes the named value - or the whole key when name is omitted - during installation.

Shortcuts: [shortcut.ID]

```
[dir.Menu]
path = "Programs/Example"    # Start menu > Example

[shortcut.Settings]
dir = "Menu"
name = "Example settings"    # ".lnk" is added
target = "file:MainExe"
args = "--settings \"[INSTALLDIR]\""
```

A shortcut belongs to its target file (and its feature); folders created for it are removed at uninstall. In a per-machine package Programs and Desktop are the all-users Start menu and the Public Desktop.

In an MSIX a shortcut starts an application (its target must be an [msix-app.*] executable); one in Programs or StartMenu is that application's own Start menu entry (no args); one on the Desktop is written into the manifest and needs min-version = "10.0.19645.0" or later (RP1614). Other folders, working-dir and [...] in args are errors there (RP1613); for Startup use a startup task.

File types and links: [assoc.ID], [protocol.ID]

```
[assoc.Doc]
extension = ".exdoc"
prog-id = "Example.Document"      # tables sharing it describe one kind of document
description = "Example document"
target = "file:MainExe"
args = "--open \"%1\""           # the default is "%1"

[protocol.Link]
name = "example"                  # example:... opens the program
target = "file:MainExe"
```

In an MSI they are registry values under HKEY_CLASSES_ROOT, in the program's component: the extension's default value is the prog-id, the prog-id has the description, DefaultIcon (icon, or the program's first icon) and shell\open\command; a scheme gets URL Protocol. HKEY_CLASSES_ROOT follows the installation: per machine they land in HKLM\Software\Classes, per user in HKCU\Software\Classes, and removal takes them away. A type that only this program claims opens in it directly; where the user has chosen another program, Windows keeps that choice.

In an MSIX they go into the manifest of the application whose executable is the target (a file type association, a protocol), and args must be plain text. icon is not used there: the application's logo stands for the file type.

MSIX only: [msix-extension.ID]

```
[msix-extension.Cli]
kind = "alias"
alias = "example.exe"             # typed in a console, it starts the application

[msix-extension.Boot]
kind = "startup-task"             # starts with Windows once the application has run once
display-name = "Example"          # the name in Task Manager; task-id defaults to the table ID
enabled = true
```

An MSI build leaves these out. Fonts ([font.*]) in an MSIX are shared with other applications (uap4:SharedFonts) from the package's Fonts folder; title is not used there.

Removing and copying: [remove.ID], [copy.ID]

[remove.ID] deletes files matching name in dir - for example *.log an older version left, at on = "install", or files the program writes at run time, at on = "uninstall". Without name it removes the folder itself when it is empty. A failed installation puts removed files back. [copy.ID] installs a second copy of a file of this package into another folder; it goes and comes with its source. Folders that existed before the installation are never removed.

Environment variables: [env.ID]

A system (machine-wide) variable. mode = "set" (default) replaces it, append/prepend add ;value to the end or value; to the front of what is there. Uninstall undoes exactly that: a set variable is deleted, an appended part is taken out and the rest kept. keep = true leaves it.

INI files: [ini.ID]

mode = "set" (default) writes key=value in [section] of the file, add appends the value to a comma-separated list (a becomes a,b), remove deletes the key during installation. Uninstall takes out what set and add wrote. remove runs before files are installed, so it is for INI files an older version left behind.

Searching and requiring: [search.ID], [require.ID]

A search runs before anything else and puts what it found into a public property (empty when nothing is found): a registry value's data, the full path of a file (path = a known folder and a relative path, like System or ProgramFiles/Example; min-version for program files), a folder, or the key file of another product's component. A requirement stops a first installation with its message when its condition is false (repair and removal are never blocked); conditions use Windows Installer's syntax (VersionNT >= 603, FOUND_TOOL, NOT OLDSETTING) and may test search results.

```
[search.Tool]
property = "FOUND_TOOL"
kind = "file"
path = "System"
file = "tool.exe"

[require.Tool]
condition = "FOUND_TOOL"
message = "[ProductName] needs tool.exe."
```

A registry or dir search may name a dir instead of a property: the folder it finds becomes that dir's default. This is how a package remembers the folder the user chose: write the folder down, and look it up in the next version. A registry value is used only when that folder still exists; a folder given on the command line (msiexec /i app.msi INSTALLDIR=D:\Apps\Example\) still wins.

```
[registry.RememberDir]
root = "HKLM"
key = "Software\\Example"
name = "InstallDir"
value = "[INSTALLDIR]"

[search.PreviousDir]
property = "INSTALLDIR"           # a dir: its default, when the registry holds an existing
folder
kind = "registry"
root = "HKLM"
key = "Software\\Example"
name = "InstallDir"
```

Services, fonts and permissions

[service.ID] installs a service run by an .exe of the package: it is stopped before its files change and at uninstall, deleted at uninstall, and started after installation with start-on-install = true. [font.ID] registers a font file that the package installs into the Fonts folder; without title Windows reads the name from the TrueType/OpenType file. [permission.ID] sets an SDDL security descriptor on a folder the package creates, one of its files, or a registry value it writes (this raises the package to Windows Installer 5.0).

Per-user and dual packages: scope

scope = "machine" (default) installs for everyone and needs administrator rights. scope = "user" installs for the current user without elevation: ProgramFiles becomes %LOCALAPPDATA%\Programs, Programs and Desktop are the user's, registry values go to HKCU (or HKMU), environment variables are

the user's, and do/undo actions run as the user; asking for a per-machine installation is refused. `scope = "dual"` installs per user by default and per machine from an elevated prompt with `msiexec /i x.msi ALLUSERS=1 MSIINSTALLPERUSER=""`; its registry values use HKMU, which is HKLM or HKCU as installed. Services, fonts, permissions and the machine folders (Windows, System, Fonts, CommonAppData) need `scope = "machine"`.

Cabinets

Files are compressed into one cabinet embedded in the package. `cab-max-size = N` starts a new cabinet after N MiB of files; `cab = "external"` writes the cabinets next to the package as `<name>.cab` (or `<name>-1.cab`, `<name>-2.cab`, ...), which must travel with it; without `cab-max-size` external cabinets are split before 2 GiB each. Windows Installer cannot open a package of 2 GiB or more, so an embedded cabinet that large is an error (RP1516) that asks for `cab = "external"`. rubrapack never overwrites an existing cabinet and writes the package last: it is built in `<out>.rp-map` (the package's bytes go straight into that file, not into memory) and renamed to its name only when everything succeeded, so a failed build leaves no package. Compression runs on every processor (`--jobs N` to use fewer); the bytes are the same either way. Every package also carries the administrative (`msiexec /a`, an uncompressed network image) and advertisement (`msiexec /jm`) sequences.

Only these fields are MSI formatted strings: registry value (and multi items), shortcut args, environment value, INI value, requirement message, service args. Everywhere else rubrapack writes the text exactly as given.

Installed apps entry and properties

[arp] sets how the product appears in Settings > Installed apps (no-modify, no-repair, help, about). [property.NAME] adds a public property; `secure = true` lets it reach the elevated part of the installation, `hidden = true` keeps its value out of logs. Names the installer or rubrapack set themselves (ARP*, MSI*, RP_*, ALLUSERS, REBOOT, ...) are refused.

Installing from the command line

Everything the dialogs choose can be given to `msiexec` instead:

Property	What it does
INSTALLDIR=D:\Apps\Example\	the install folder (any dir with an upper-case ID)
ADDLOCAL=Core,Extra	install these features (ADDLOCAL=ALL: every feature)
REMOVE=Extra	remove these features from an installed product (REMOVE=ALL: everything)
INSTALLLEVEL=3	install every feature whose level is at most 3
RPLANGUAGE=ko	the dialogs' language (with [ui] languages)
ALLUSERS=1 MSIINSTALLPERUSER=""	a dual package for everyone (default: just the current user)
DESK=1, APP_MODE=server	your own properties: when conditions, dialog values

Installing without a window

Every package rubrapack builds installs, repairs, upgrades and uninstalls from the command line with no user interface:

```
msiexec /i example.msi /qn /l*v install.log
msiexec /x {ProductCode} /qn
```

Exit code 0 is success, 3010 success with a restart needed (rubrapack never restarts the machine itself), anything else a failure after which the machine is as before.

Dialogs: ui

ui in [package] picks one of the built-in dialog sets. Without it (none) the package shows only Windows Installer's own progress bar.

ui	Installing	Already installed
basic	progress, then finished (or error)	progress, finished
minimal	welcome, the license if there is one, progress, finished	repair or remove
installdir	welcome, license, install folder (with a folder browser), ready, progress, finished	repair or remove
features	as installdir, plus a feature tree and the disk space it needs	repair or remove

Every set also has the cancel question, the error dialog, the files-in-use list and the out-of-disk-space warning. The dialogs only collect values that all have defaults, so /qn still installs without a window.

```
[package]
ui = "installdir"
license = "LICENSE.txt"           # the Install/Next button stays disabled until it is accepted

[ui]
install-dir = "APPDIR"           # the dir the user may change (default INSTALLDIR)
banner = "banner.bmp"           # the strip at the top; about 493 x 58 pixels

[ui-text.WelcomeText]
text = "This will install [ProductName]. Close other programs first."
```

A .txt or .md license is shown as plain text (Korean, emoji and any other characters are kept); an .rtf license is used as it is. Without banner the strip is plain white. [ui-text.ID] replaces one text; the texts are MSI formatted strings, so [ProductName] is replaced and a literal [is written [\]. The IDs are: Back, Next, Cancel, Install, Finish, OK, Yes, No, Retry, Ignore, Abort, Exit, Browse, WelcomeTitle, WelcomeText, LicenseTitle, LicenseText, LicenseAccept, DirTitle, DirText, DirLabel, BrowseTitle, BrowseText, BrowseLookIn, BrowseFolder, BrowseUp, BrowseNew, CustomizeTitle, CustomizeText, Reset, DiskCost, DiskCostTitle, DiskCostText, ReadyTitle, ReadyText, ProgressTitle, ProgressText, ProgressStatus, ExitTitle, ExitText, UserExitTitle, UserExitText, FatalTitle, FatalText, CancelText, FilesInUseTitle, FilesInUseText, OutOfDiskTitle, OutOfDiskText, MaintTitle, MaintText, Repair, RepairText, Remove, RemoveText, LanguageTitle, LanguageText, DirGuardText (the guard's message below). In button texts & marks the access key (&Next is Alt+N).

Several languages

The dialogs are in English. [ui] languages adds other languages to the same package; then the first page asks for the language, and every page after it - welcome, license, folder, your own pages, ready, progress, finished, and the cancel, error, files-in-use, disk-space and maintenance pages - speaks the one chosen. The choice made for the user in advance is the first added language whose LANGIDs hold the user's regional format (UserLanguageID), then the system locale (SystemLanguageID), else English. RPLANGUAGE=ko on the command line chooses directly; a silent installation (/qn) shows nothing and needs no choice. With English alone there is no language page.

```
[ui]
languages = ["ko"]               # English is always there, and the default
license-ko = "LICENSE-ko.txt"    # a license per language (default: [package] license)

[ui-text.WelcomeText]
text-ko = "[ProductName]을(를) 설치합니다."  # text = every language, text-xx = one
```

```
[dialog.Options]
after = "RpInstallDirDlg"
title = "Options"
title-ko = "선택 사항"

[dialog-control.Mode]
# ...
labels = ["&Typical", "&Portable"]
labels-ko = ["표준(&T)", "휴대용(&P)"]
```

- The built-in texts exist in English and Korean (ko). Another language (ja, de, ...) gives every built-in text as text-xx (lint names the ones missing), and may give name-xx (its name on the language page), font-xx (the face of its dialogs) and langid-xx (the LANGIDs that choose it: one number or an array). Common languages have these built in: ja, zh, de, fr, es, it, pt, nl, pl, ru, uk, tr, vi, th.
- Korean dialogs use Malgun Gothic, English ones Segoe UI.
- A text that uses a property other than [ProductName], [Manufacturer] and [ProductVersion] is formatted when the language is chosen, and must fit 255 characters.
- Windows Installer's own texts - the feature tree's menu, sizes, the time left, and its error messages - are not the package's: the first stay English, the error messages follow Windows.
- [package] language sets only the package's language (ProductLanguage, the summary). Since 0.2 it no longer makes the dialogs Korean: add languages = ["ko"] (lint warns, RP1317).

Icon, the finished page, and the scope page

- [arp] icon = "app.ico" is the product's icon in Installed apps; [shortcut] icon gives a shortcut its own .ico (without it, the program's own icon). Both are read when the package is built and stored in it.
- [ui] launch = "file:App" puts "Launch [ProductName]" on the finished page (ticked unless launch-checked = false; launch-args are its arguments). Finish starts the program after a first installation or an upgrade, as the user who ran the setup, not with the installer's rights; not after a repair or removal, and never at /qn.
- scope = "dual" with dialogs adds a page after the license: "Just me" (the default) or "Everyone on this computer", which needs administrator rights; the install folder moves to %LOCALAPPDATA%\Programs or Program Files accordingly.

Your own dialog pages: [dialog.ID], [dialog-control.ID]

With minimal, installldir or features you can add pages to the built-in flow. A page gets the same banner and Back / Next / Cancel buttons as the others; you place the controls in its body.

```
[dialog.Options]
title = "Options"                # the banner heading; [ProductName] when omitted
description = "Choose how to install."
after = "RpInstallDirDlg"        # shown right after this page

[dialog-control.ModeLabel]
dialog = "Options"
type = "text"
x = 20
y = 55
width = 330
height = 12
text = "Installation &mode:"

[dialog-control.Mode]
dialog = "Options"
type = "radio"
```

```

x = 20
y = 70
width = 200
height = 42          # at least 12 per value
property = "APP_MODE"
values = ["typical", "portable", "server"]
labels = ["&Typical", "&Portable", "&Server"]

[property.APP_MODE]
value = "typical"    # the default, also for a silent installation

```

- after is a page of the chosen set - RpWelcomeDlg, RpLicenseDlg (with a license), RpInstallDirDlg (installdir, features), RpCustomizeDlg (features) - or another of your pages. Several pages after the same page follow in ID order. In minimal the last page's button is Install; in the other sets the ready page always comes last.
- Coordinates are dialog units on a 370 x 270 page; controls go in the body, between y = 45 and y = 234. The Tab key moves through the controls from top to bottom, then left to right.
- text shows a label (& marks an access key; on a text control it moves to the next control). checkbox sets its property to 1 when ticked and removes it when clear. edit lets the user type the property's value. radio and combo choose one of values (1 to 32 strings of at most 64 bytes), shown as labels (default: the values themselves), in the order written.
- A radio or combo needs a [property.*] whose value is one of its values: the dialogs only collect values, and a silent installation (/qn) uses the defaults. Any property can also be set on the command line: `msiexec /i example.msi /qn APP_MODE=server`.
- The properties are public names of your own (upper case), one control each, and reach the elevated part of the installation, so [registry.*], [ini.*], [env.*], conditions and the rest can use them as [APP_MODE].
- Page IDs starting with Rp and the control IDs of the frame (Banner, Title, Description, BannerLine, BottomLine, Back, Next, Cancel) are reserved; a page holds up to 64 controls.

Architectures and upgrade families

arch is the source's own architecture and upgrade-code its upgrade family. To build the same source for another architecture (--arch), give that architecture its own family with upgrade-code-x86, upgrade-code-arm64 or upgrade-code-x64; without it the build is refused. Windows Installer's upgrade detection cannot tell architectures apart, so sharing one code would make installing one architecture remove the other. An MSIX has no upgrade code, so .msix and .msixbundle builds do not need these.

Versions that must be removed first

refuse-upgrade-below = "1.0.0" refuses to upgrade an installed version below 1.0.0 and tells the user to remove it first; the message (refuse-upgrade-message, or a default one) always ends with the command that does it: `msiexec /x {ProductCode} /qn MSIRESTARTMANAGERCONTROL=Disable`. Use it when older versions were built by another tool without MSIRESTARTMANAGERCONTROL=Disable: removing such a version inside an upgrade would try to close every program that has its files loaded (see formats/msi-package.md, "Files in use").

Wildcards: [files.ID]

glob is a source path with * (any characters within one folder), ? (one character) and ** (any number of folders). Matches are sorted by name, so the result never depends on the file system. Folders below the first wildcard are recreated under dir: `glob = "dist/layouts/**/* .jmt"` installs `dist/layouts/de/x.jmt` as `<dir>/de/x.jmt`. No match, a symbolic link, or the output file itself among the matches is an error.

Empty folders: [folder.ID]

Creates the folder name inside `dir` even when no file goes there. With `keep = true` the folder stays after uninstall (for data the program writes).

Features

Without any [feature.*] table everything goes into one hidden feature. As soon as one feature is declared, every file needs one: its own feature key, or the feature of its dir. A feature with a level above 1 is not installed by default. With `ui = "features"` the user picks features in a tree, and changes them later from Installed apps (Change on the maintenance page).

- `required = true`: the tree offers no "will be unavailable" for it.
- `follow-parent = true`: installed wherever its parent is (on this computer, or not at all).
- `when = "<condition>"`: the feature is off - not installed and not shown - unless the condition holds (see below).

Conditions: when

`when` takes a Windows Installer condition (`VersionNT64`, `DESK = "1"`, `NOT OLDVERSION`) on a feature, a file or file group, a registry value (not one with a file: put it on the file), a shortcut, an environment variable or an INI value. What it guards is installed only when the condition holds; properties set in your own dialog pages, on the command line or by a search can be tested. The condition is evaluated when the item is first installed, a major upgrade included (a repair keeps what is there); an MSIX installs everything and refuses when.

```
[dialog-control.Desk]                # a check box in your own page (see below)
dialog = "Options"
type = "checkbox"
x = 20
y = 60
width = 300
height = 16
text = "Create a &desktop shortcut"
property = "DESK"

[shortcut.Desk]
dir = "Desktop"
name = "Example"
target = "file:App"
when = "DESK"
```

Variables

`$(NAME)` inside a string value is replaced once, from `-D NAME=value` on the command line first, then [define]. The result is not read again (a value containing `$(X)` stays literal). `$$` is a literal `$`. An undefined name is an error. `$(ARCH)` is built in, unless `-D` or [define] defines it: the architecture being built (x64, arm64 or x86) - so one source can name each architecture's program, as `source = "bin/$(ARCH)/app.exe"`.

Program files

Files that are Portable Executables (`.exe`, `.dll`, ...) are checked: their machine type must match arch (an x86 helper in an x64 package needs `any-arch = true`), and their version resource becomes the file's version in the package, which is how Windows Installer decides whether to replace an installed file.

Leaving a file at removal: keep

`keep = true` on a file (or a file group) leaves it in place when the product is removed - for settings the user may have changed. A repair or a later version does not overwrite such a file once it has changed (Windows Installer's rule for files without a version). An MSIX removes all its files and refuses `keep`.

Guarding the install folder

A program whose files are loaded by other programs - an input method, a shell extension, a service - must not be installed into a folder that someone else prepared: files already there stay, and a DLL planted beside the program would be loaded with its rights. `guard = true` on a dir makes a first installation stop before any file is placed when that folder

- already exists and its owner is not SYSTEM, Administrators or TrustedInstaller, or
- is reached through a junction or another reparse point (the folder itself or any folder above it).

A folder that does not exist yet passes (the installer creates it; `[permission.*]` can lock it). Repair, upgrade-in-place and removal are not checked; a major upgrade is a first installation of the new version and is checked like one (the folder the earlier version created passes). The installation ends with 1603 after the message `DirGuardText ([1]` is the folder; in the language chosen in the dialogs), also at `/qn`, and the log names the owner found. The check is done by rubrapack's helper DLL, which the package then carries (as for `REG_QWORD` values). What is checked is the owner: a folder owned by Administrators whose permissions let anyone write is not refused (lock it with `[permission.*]`).

```
[dir.INSTALLDIR]
path = "ProgramFiles/Example"
guard = true
```

Paths

Source paths are relative to the source file and use `/`. Absolute paths, `\`, symbolic links and missing files are errors. Target names may use any Unicode text except what Windows forbids (`< > : " / \ | ? *`, control characters, trailing dot or space, device names such as `CON`), and two names in one folder may not differ only by letter case. Windows Installer stores a name together with its 8.3 short name in 255 UTF-16 units, so a long name may use only what the short name leaves: 242 units for `name.ext` with a three-letter extension, 246 for a name without one (NTFS would accept 255). A longer name is refused with `RP1514`.

4.5 Command line

```
rubrapack build <src.toml> -o <out.msi|out.msix|out.msixbundle> [-D NAME=VALUE]...
    [--arch x64|arm64|x86 | --arch <list> (.msixbundle)]
    [--compress none|mszip|mszip:N] [--jobs N] [--nfc] [--reproducible]
    [<key> [--cert <chain.pem>]
    [--timestamp <URL> [--tsa-trust <certificates>] [--tls-trust <certificates>]
    [--system-roots]
    [--proxy <URL>]] [--allow-unsigned-cabs]]
    [--unsigned-test] [--msix-compress deflate|store] (.msix, .msixbundle)
rubrapack inspect <file.msi> [table | --summary | --files | --streams]
rubrapack inspect <file.msix|file.msixbundle> [--files | --manifest]
rubrapack inspect <file.cab>
rubrapack new [msi] <name>
rubrapack new [<file>.toml] [-i]
rubrapack new <name> [--dist <folder>] [--name ...] [--ui ...] [--optional ...] ...
rubrapack edit <file>.toml [--set <table>.<key>=<value>]... [--unset <table>.<key>]... [--sync]
rubrapack guid [--from <text>]
rubrapack lint <src.toml> [-D NAME=VALUE]... [--arch x64|arm64|x86] [--target msi|msix] [--nfc]
    [--strict]
```

```

rubrapack lint <file.msi> [--previous <old.msi>] [--strict]
rubrapack lint <file.msix|file.msixbundle> [--strict]
rubrapack extract <file.msi|file.msix|file.msixbundle|file.cab> -d <new dir> [--limit-entries
N] [--limit-bytes N]
rubrapack sign <file.exe|.dll|.msi|.msix|.msixbundle> <key> [--cert <chain.pem>]
                [--timestamp <URL>] [--tsa-trust <certificates>] [--tls-trust <certificates>] [--
system-roots]
                [--proxy <URL>]] [--allow-unsigned-cabs] [-o <out>]
rubrapack keys list [--pkcs11 <module>] [--token-label <label>] [--pin-env VAR | --pin-file
FILE]]
rubrapack verify <file.exe|.dll|.msi|.msix|.msixbundle> [--trust <certificates>]... [--system-
roots] [--tsa-trust <certificates>]...
rubrapack version | help [command]

<key> is one of:
  --key <key.pfx|.pem> [--pass-env VAR | --pass-file FILE]                a key file
  --pkcs11 <module> --key-label <label> [--token-label <label>]
    [--pin-env VAR | --pin-file FILE]                                     a key in a
PKCS#11 token
  --key-store <SHA-1 thumbprint> [--machine-store]                       a key in the
Windows store

```

- Command-line options override the source.
- `--reproducible` derives the package code from the content: the same source gives the same bytes, on Linux and on Windows, from any folder, with the options in any order. Without it the package code is random, as Windows Installer expects for different package files. An MSI holds no time at all (no creation or save dates in its summary; cabinet entries are dated 1980-01-01), so `SOURCE_DATE_EPOCH` does not change it; nor does it hold the source's path or the name of the machine or user that built it. Every source file is read once, and those bytes are hashed, versioned and packed; a file that changes while the package is being built stops the build (RP1515).
- `--nfc` puts the names the package gives to folders, files and shortcuts in Unicode NFC (composed). A file from macOS often has a decomposed name - a Hangul syllable such as U+D55C as the three jamo U+1112 U+1161 U+11AB - and Windows installs names exactly as they are, so the same word can become two different files. The source files keep their names; texts and registry values stay as written. `lint` warns (RP2105) about any text that is not in NFC, and a name that becomes the same as another one is refused (RP1511). The normalization is rubrapack's own, from Unicode 17.0 data.
- `inspect <file.msi> <table>` prints the table in Windows Installer's IDT format. `--files` prints one tab-separated line per file: installed path, size, File key, component, version, language, MD5 (from `MsiFileHash`; empty when the package has none). `--streams` lists the streams and their sizes. `inspect <file.cab>` lists the files in a cabinet. `inspect <file.msix>` shows the identity, the executable and the files (after checking every block's hash); `--files` lists path, size and whether each file is compressed, `--manifest` prints `AppxManifest.xml`. For a bundle, the identity and the packages (each opened and checked as a package), and its `AppxBundleManifest.xml`; `extract` writes the packages out.
- `new <name>` writes `<name>.toml`, a source that builds as soon as the program's files are in `dist/`, with a fresh upgrade-code. `new <file>.toml`, `new` without a name (which asks for the file name too), or `new <name> -i` asks the questions of A first package instead (on `stderr`; answers from `stdin`, one per line, so they can be piped; the input ending before the last answer writes nothing), and `new <name>` with options takes the same answers without asking: `--name`, `--manufacturer`, `--version`, `--dist` (default `dist`), `--main <file>|-`, `--arch`, `--install-dir`, `--scope`, `--optional <folder,...>|-`, `--ui`, `--license <file>|-`, `--languages ko|-`, `--shortcuts start,desktop|none`; what is left out takes the default the question would offer. The source lists the folder's top-level files one by one (a shortcut names a file, and a glob cannot leave one out) and each sub folder

that holds files as a glob; optional sub folders become features at level = 2 next to a required Main. It is checked like lint after it is written (exit 1 if that finds a problem). It never replaces an existing file; new app.toml over an existing app.toml says to use edit.

- `edit <file>.toml` changes a source in place. Without options it shows a menu: 1 name, manufacturer, version (in [define] VERSION when version = "\$ (VERSION)") and architecture; 2 the folder under Program Files and the scope; 3 dialogs, license and Korean dialogs; 4 which sub folders are optional parts (the first one adds a required Main feature and gives it to everything that then needs a feature); 5 the Start menu and desktop shortcuts and the program they open; 6 the file list against the program folder - files and sub folders that are gone are removed, new ones added; 7 any table.key; v shows the text, l checks it like lint, s saves and checks, q leaves (asking first when something changed). Each question offers the current value. Only the values changed are rewritten - comments, order and tables written by hand stay; a change that would not parse is not made. With options it asks nothing, applies them in order and saves: --set package.version=1.2.0 (a value that is TOML - "text", a number, true, ["ko"] - is used as it is, anything else as a string; a table that is not there is added), --unset package.license, --sync (6 without questions). Sources in UTF-16 are refused: save them as UTF-8 first.
- `guid` prints a random GUID (version 4). `guid --from <text>` prints the GUID rubrapack derives from a text, the same way on every machine: SHA-256 over the 32-bit little-endian length of guid and those 4 bytes, the 32-bit little-endian number 1, and the 32-bit little-endian length of the text and its UTF-8 bytes; the first 16 bytes of the hash, with the version nibble set to 8 and the variant bits to 10 (RFC 9562 UUIDv8), written as {XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX} in upper case. For example `guid --from hello` gives {B52FE8EF-B68D-84B5-91AF-E6E01BEC2773}.
- Diagnostics look like `example.toml:12:3: error[RP1201]: unknown key 'naae' in [package] (did you mean 'name'?)`.
- Before writing, `build` checks the finished tables (RP20xx diagnostics, exit code 5); these checks guard rubrapack itself, so a source that passes the RP1xxx checks should never meet them.
- `lint <src.toml>` runs every check `build` runs and writes nothing. It checks the source for an MSI; --target msix checks it for an MSIX instead (the [msix] tables, and what an MSIX cannot carry, such as keep or when). `lint <file.msi>` checks a package made by any tool with the same table rules: what stops an installation is an error (exit code 5) - a value that does not fit its column, a missing referenced row, a duplicate key, a broken dialog tab order (RP2101), a dialog's first/default/cancel control that is not there (RP2102), a files-in-use dialog without a ListBox table (RP2103), an error dialog without ErrorText/ErrorIcon (RP2104) - and everything else is a warning. --strict makes warnings fail too. A package in another code page than 65001 gets a note (RP2100): its text cannot be checked as UTF-8. The last line on stdout counts errors and warnings. `lint <file.msix>` checks the package the way Windows reads it - the ZIP, the block map, every block's hash - and that the manifest has an identity and names files that are in the package (RP2201, RP2202). For a bundle: its manifest's block map, and for each package where the manifest says it lies, its size, its identity and the package itself; one package per architecture.
- `lint new.msi --previous old.msi` also checks that new.msi upgrades old.msi cleanly. Errors: another UpgradeCode (RP2301: the new package does not replace the old one), a version that is not higher in its first three fields (RP2302: Windows compares only those). Warnings: the same ProductCode (RP2303: an upgrade needs a new one), a component whose GUID stays but whose key path (file, folder or registry value) changed (RP2304), or whose 64-bit flag changed (RP2305), a component that is gone (RP2306: its resources are removed with the old version), a feature that is gone (RP2307: a patch or a change of the installed set loses it). The last line names the previous package.
- `extract` unpacks a package the way it installs: folders by their long names under the Directory tree (a standard folder such as ProgramFiles64Folder keeps its name), files from the embedded or external cabinets, or from the source folders next to an uncompressed package. Every file is

checked against its size and `MsiFileHash`. The target must be new or empty, and nothing is written until the whole package has passed: names with `..`, `/`, `\`, `:`, a drive, a reserved device name (`CON`, `COM1`, ...), a trailing dot or space, or two paths that differ only in case are refused. The defaults allow 100,000 entries and 16 GiB. A `.cab` unpacks by the names inside it. An `.msix` unpacks its files (not the ZIP's own `[Content_Types].xml` and `AppxBlockMap.xml`) after every block has matched its hash.

- `sign` adds an Authenticode signature (SHA-256; RSA, or ECDSA P-256/P-384) to a PE file, an MSI package, an MSIX package or a bundle (whose packages are signed first), in place or to `-o`; `build --key` signs the package as it is built (the same code), and a signed `--reproducible` build still gives the same bytes every time (the signature holds no time) - unless it is timestamped: a timestamp carries the server's time and serial number, so it differs on every run, and only the package inside the signature stays the same. An MSI whose cabinets lie outside it is refused: its signature would not cover them - use embedded cabinets, or `--allow-unsigned-cabs` to sign the `.msi` alone with a warning. The key file is a PKCS#12 file (`.pfx/.p12`: AES with PBKDF2-HMAC-SHA256, SHA-256 MAC - what current Windows and OpenSSL export) or a PKCS#8 PEM/DER key, plain or encrypted with PBES2; `--cert` adds certificates the key file does not hold. The certificate must be for code signing, must not be a CA and must be valid now. The password comes from an environment variable or a file, never from the command line. A file that is signed already is refused. An MSIX's `[msix] publisher` must be the certificate's subject as Windows writes it - its parts from last to first, as `O=Example Ltd, CN=Example` - or signing stops and prints the subject to use; `--unsigned-test` and a key do not go together.
- A key that must not leave its hardware - required for publicly trusted code-signing certificates since 2023 - stays there: rubrapack computes the hashes, the CMS structure and the timestamp request itself and asks only for the one signature. `--pkcs11 <module>` loads the token's PKCS#11 library (the only library rubrapack ever loads, and only this one you name), logs in with the PIN from `--pin-env` or `--pin-file` (never the command line), and uses the private key labelled `--key-label` with the certificate stored under the same ID on the token (`--cert` supplies it, or the rest of the chain, when the token has none); with several tokens present, `--token-label` picks one. On Windows, `--key-store <thumbprint>` uses a certificate in the current user's personal store (`--machine-store`: the local machine's) whose private key is reachable through NCrypt - a smart card, a token or TPM through its key storage provider, or a software key - and takes the chain Windows builds for it. RSA and ECDSA keys both work. Every signature is checked with the certificate before it is written.
- `keys list` shows the keys that can sign: with `--pkcs11`, each private key on the token(s) with its label, ID and certificate; on Windows without it, the certificates with a private key in the personal stores, with their thumbprints. For each it prints the MSIX publisher that certificate signs for and when it expires; nothing secret.
- `--timestamp <URL>` asks an RFC 3161 time-stamping server to countersign the signature, so that it stays valid after the certificate expires; without it `sign` and `build --key` warn that it will not. There is no default server and nothing goes over the network unless you name one. Public servers include `http://timestamp.digicert.com`, `http://timestamp.sectigo.com` and `http://time.certum.pl`; http is fine, because the answer is itself signed and checked (the nonce, the hash of the signature, the server's signature and its time-stamping certificate). `--tsa-trust` also requires the server's certificate path to end in one of the given certificates. For an https server rubrapack speaks TLS 1.3 itself (AES-GCM, x25519 or P-256, RSA-PSS or ECDSA server keys; not TLS 1.2) and checks the server's certificate - its path to a root given with `--tls-trust` or found in the operating system's store with `--system-roots`, its dates, that it names the server (subjectAltName; no wildcard across dots) and that it is for TLS servers; nothing turns these checks off. The three trusts stay apart: `--tls-trust` and `--system-roots` say whom to believe about the connection, `--tsa-trust` whom to believe about the time; neither is used for the other. On Linux `--system-`

roots reads `$SSL_CERT_FILE` or the distribution's CA bundle; on Windows the ROOT store, which Windows fills with some roots only when something first needs them, so a root may be missing there until then. `--proxy http://host:port` sends the request through a proxy (an https server through CONNECT, so the proxy sees only the server's name); without it `https_proxy` or `HTTPS_PROXY` (for an https server) or `http_proxy` (for http; the upper-case `HTTP_PROXY` is not read, since a CGI environment can set it from a request header) are used unless `no_proxy` or `NO_PROXY` - names, each also covering the names under it, or `*` - lists the server. A proxy that asks for a password is not supported (a clear error), nor an `https://` proxy. When the timestamp fails - no answer, a refusal, a bad answer, a time outside the signing certificate's validity - nothing is signed and nothing is written (exit code 6); the signature never quietly goes out without it. The limits: 10 s to connect, 60 s in all, a 4 MiB answer, 3 redirects, 2 retries after a network error or a 5xx answer.

- `verify` prints what it checked - structure, digest, signature, the path to a certificate given with `--trust`, revocation (never looked up: not-checked) and timestamp - and succeeds only when all of them hold and the path ends in a trusted certificate. Without `--trust` it does not call anything trusted (exit code 4); `--system-roots` adds the operating system's roots to `--trust` for the signer's path (not for the timestamp's). It says nothing about Windows' own reputation checks. The timestamp line is not-present, `<time>`, TSA not-checked (give `--tsa-trust`), `<time>`, TSA trusted, `<time>`, TSA untrusted, invalid or unsupported (the older Authenticode timestamp that Set-AuthenticodeSignature -TimestampServer writes, which rubrapack does not check); the last three fail. Only a timestamp whose server is trusted through `--tsa-trust` - `--trust` does not count for it - moves the time at which the signer's certificate must have been valid from now to the stamped time. (For tests, the environment variable `RUBRAPACK_TEST_NOW` - seconds since 1970 - replaces "now" in `verify`'s checks, so that an expired certificate can be checked without changing a clock; it widens nothing that a clock set to that time would not.)
- Exit codes: 0 success, 1 error in the source, 2 usage, 3 input/output, 4 signing, 5 lint, 6 network.

4.6 Diagnostic codes

Every problem rubrapack reports has a code, `error[RPnnnn]` or `warning[RPnnnn]`, after the file, line and column it is about. The first two digits say what kind of problem it is:

Codes	What went wrong	Where to look
RP00xx	the command line: an unknown command or option, a file that cannot be read or written	<code>rubrapack help <command></code>
RP10xx	the source file's encoding: not UTF-8 (or UTF-16 with a BOM), stray carriage returns	save the file as UTF-8
RP11xx	TOML outside the subset rubrapack reads: multi-line strings, inline tables, a table defined twice	The TOML subset
RP12xx	tables and keys: an unknown table or key (with a suggestion), a required key or table missing, something without a feature once features exist	Tables
RP13xx	values: IDs (unique across all tables, not reserved), GUIDs, versions, numbers out of range, references to things that do not exist	the table's section
RP14xx	variables: <code>\$(NAME)</code> without a value, <code>\$(</code> not closed	Variables
RP15xx	the files to install: not found, a directory, a link, a glob without a match, a program for another architecture, names too long, a file that changed during the build, a package too large	Paths, Program files
RP16xx	MSIX: what an MSIX package needs, and what it cannot carry	MSIX packages
RP19xx	a feature this rubrapack does not provide	-
RP20xx	the finished tables break a Windows Installer rule. A source that passes the <code>RP1xxx</code> checks should never meet these: please report it	-

Codes	What went wrong	Where to look
RP21xx	lint of a package: dialogs, code page, text normalisation	lint under Command line
RP22xx	lint of an MSIX package or bundle	lint under Command line
RP23xx	lint --previous: this package would not upgrade the previous one cleanly	Versions and upgrades

The message says what to change; the codes above are the ones to search the manual for.

Part III

File formats

5 Package formats for implementers

In this chapter

5.1	How to read the facts	35
5.2	Conventions	35
5.3	The pipeline, in one picture	35

These pages describe what a program has to write to produce a Windows Installer package (.msi) and an MSIX package or bundle that Windows accepts, and how to sign them. They are written from public specifications and from experiments against Windows itself, so that you can build your own tool from them without reading rubrapack's code - or check rubrapack's code against them.

Page	Covers
Compound File Binary (CFB)	Compound File Binary: the container every .msi is
The MSI database inside the compound file	Streams, string pool, system tables, table encoding, IDT export
Summary information	The summary information stream and its code page trap
The smallest package that installs, repairs, upgrades and uninstalls	The tables that install, upgrade, repair and uninstall: files, registry, shortcuts, services, fonts, dialogs, per-user packages, sequences
Program files: machine type and version	Program files: machine type and version resource
Cabinets, MSZIP and deflate	Cabinet files, MSZIP blocks, the deflate encoder
Deterministic identities and reproducible packages	Deterministic GUIDs and keys, reproducible builds
Authenticode signatures	Authenticode signatures for PE files and MSI packages: the digests, the CMS structure Windows accepts, ECDSA, RFC 3161 timestamps
Checking your output against Windows	How to check your output with Windows' own components
MSIX packages	MSIX: the ZIP layout, block map, manifest, virtual registry and file system, extensions, bundles, signatures

Page	Covers
Registry hive files (REGF)	Registry hive files (REGF), as MSIX Registry.dat and User.dat hold them

5.1 How to read the facts

Every statement is tagged with where it comes from:

- **[spec]** - a public specification: [MS-CFB], [MS-OLEPS], [MS-CAB], [MS-MCI], RFC 1951, RFC 1321, RFC 3161, RFC 5652, RFC 9562, the PKWARE ZIP APPNOTE, ECMA-376 Part 2 (OPC), the Authenticode PE specification, or the Windows Installer and MSIX references on Microsoft Learn.
- **[observed]** - measured on Windows 11: the MSI pages on 25H2 (build 26200) with Windows Installer 5.0.10011, by creating databases through `msi.dll` and reading them back byte for byte, and by installing packages; the MSIX, registry hive and signature pages on build 26100, against Windows' own packaging API, Offline Registry Library and signer, and by installing and running packages. Observations are facts about those builds; where Microsoft does not document a format (the MSI table encoding, the registry hive, the MSIX signature's record of hashes are not published), observation is the only source.

No text or code here comes from other MSI, MSIX, cabinet or registry implementations.

5.2 Conventions

- All integers are little-endian unless stated otherwise.
- `u16`, `u32`, `u64` are unsigned; `i16`, `i32` signed two's complement.
- Offsets are in bytes from the start of the structure named.
- Hex byte strings are written in file order: `D0 CF 11 E0`.

5.3 The pipeline, in one picture

```
source description --> tables (rows of strings/integers/streams)
                    --> MSI database encoding (string pool + one stream per table)
                    --> files packed into a cabinet (stored or MSZIP) --> one more stream
                    --> summary information stream
                    --> all streams written into one compound file = the .msi
```

An MSIX package is simpler: the payload files (each deflated in independent 64 KiB parts), `AppxManifest.xml`, `AppxBlockMap.xml` (a hash per part) and `[Content_Types].xml` in one ZIP64 archive, with `Registry.dat/User.dat` hives for registry values; a bundle stores packages side by side; a signature adds `AppxSignature.p7x` (MSIX packages, Registry hive files (REGF)).

Each arrow is a page above. None of it needs Windows to produce: `rubrapack` builds the same bytes on Linux and on Windows.

6 Compound File Binary (CFB)

In this chapter

6.1	Sectors	36
6.2	Header (first 512 bytes) [spec]	36
6.3	FAT, DIFAT, mini FAT [spec]	37
6.4	Directory [spec]	37
6.5	A simple, deterministic writer	38
6.6	Reading safely	38

An `.msi` file is a Compound File Binary file ("structured storage"): a small file system of named streams inside one file. The format is specified in [MS-CFB]; this page is the part an MSI writer needs, plus what Windows' own writer does.

6.1 Sectors

- The file is a sequence of fixed-size sectors. **Version 3** uses 512-byte sectors (sector shift 9), **version 4** uses 4096-byte sectors (shift 12). [spec]
- The header is at offset 0. Sector number N starts at file offset $(N + 1) * \text{sector_size}$ in both versions; in version 4 the header therefore occupies a whole 4096-byte sector whose bytes after the header are zero. [spec]
- `msi.dll` writes **version 4** for every new database, whatever its size (20 KB to 10 MB were tried), with minor version `0x003E`. Either version is accepted when installing; rubrapack writes version 4 by default and can write version 3. [observed]

Special sector numbers in chains and tables: [spec]

Value	Meaning
FFFFFFFFA	largest regular sector number
FFFFFFFC	DIFAT sector (in the FAT)
FFFFFFFD	FAT sector (in the FAT)
FFFFFFFE	end of chain
FFFFFFF	free sector / no stream

6.2 Header (first 512 bytes) [spec]

Offset	Size	Field	Value written
0	8	signature	D0 CF 11 E0 A1 B1 1A E1
8	16	CLSID	zero
24	2	minor version	003E
26	2	major version	3 or 4
28	2	byte order	FFFE (bytes FE FF)
30	2	sector shift	9 or 12
32	2	mini sector shift	6 (64-byte mini sectors)
34	6	reserved	zero

Offset	Size	Field	Value written
40	4	number of directory sectors	0 in version 3; the count in version 4
44	4	number of FAT sectors	
48	4	first directory sector	
52	4	transaction signature	0
56	4	mini stream cutoff	4096
60	4	first mini FAT sector	FFFFFFFFE if none
64	4	number of mini FAT sectors	
68	4	first DIFAT sector	FFFFFFFFE if none
72	4	number of DIFAT sectors	
76	436	first 109 FAT sector numbers (DIFAT)	unused entries FFFFFFFF

6.3 FAT, DIFAT, mini FAT [spec]

- The **FAT** is an array of u32, one entry per sector, giving the next sector of the chain the sector belongs to. FAT sectors themselves are marked FFFFFFFD.
- The FAT's own sector numbers are listed in the **DIFAT**: the first 109 in the header, the rest in DIFAT sectors, each holding $\text{sector_size} / 4 - 1$ numbers and, in its last u32, the next DIFAT sector (FFFFFFFFE at the end). DIFAT sectors are marked FFFFFFFC in the FAT.
- With 4096-byte sectors, 109 FAT sectors already cover about 457 MB, so real MSI files in version 4 have no DIFAT sectors. With 512-byte sectors DIFAT starts at about 7 MB.
- Streams smaller than the **cutoff (4096 bytes)** live in the **mini stream**, in 64-byte mini sectors chained through the **mini FAT** (same layout as the FAT). The mini stream is itself stored as the root directory entry's own regular stream.

The FAT must cover its own sectors and the DIFAT's; a writer computes the counts by iterating until they stop changing.

6.4 Directory [spec]

The directory is a chain of sectors holding 128-byte entries. Entry 0 is the root.

Offset	Size	Field
0	64	name, UTF-16LE, NUL-terminated (at most 31 characters + NUL)
64	2	name length in bytes, including the NUL
66	1	type: 0 unused, 1 storage, 2 stream, 5 root
67	1	colour: 0 red, 1 black
68	4	left sibling (FFFFFFFF = none)
72	4	right sibling
76	4	child (storages and root only)
80	16	CLSID
96	4	state bits
100	8	creation time
108	8	modification time
116	4	starting sector (regular or mini, by size)
120	8	stream size (in version 3 only the low 32 bits count)

- The root entry is named `Root Entry`; its starting sector and size describe the mini stream.

- The root entry of an MSI carries the CLSID {000C1084-0000-0000-C000-000000000046}, stored as bytes 84 10 0C 00 00 00 00 00 00 C0 00 00 00 00 00 00 46. [spec] [observed]
- Unused entries are all zero except the three link fields, which are FFFFFFFF. [spec]
- The children of a storage form a **red-black tree** ordered first by name length, then by comparing the upper-cased UTF-16 names. [MS-CFB] allows the simplest valid tree: every node black, the tree a plain balanced binary search tree. rubrapack builds that - a balanced tree over the sorted names, all nodes black - and `msi.dll` opens it. [spec] [observed]
- Times may be zero. Writing zero times (and no other clock input) is what makes a CFB writer deterministic. [spec]

6.5 A simple, deterministic writer

rubrapack lays the file out in this order, all runs contiguous:

```
header | FAT sectors | DIFAT sectors | directory | mini FAT | mini stream | large streams in
order
```

1. Sort stream sizes into "mini" (< 4096) and "regular".
2. Mini streams are packed back to back in 64-byte units; the mini stream length is the sum.
3. Count directory, mini FAT and mini stream sectors, then FAT and DIFAT sectors (iterate).
4. Fill FAT chains for each contiguous run, write the directory tree, mini FAT, FAT, DIFAT, header.

6.6 Reading safely

A reader must refuse, rather than follow: chains that loop (a chain never reaching FFFFFFFF within the number of sectors in the file), sector numbers beyond the file, a chain shorter than the stream size needs, directory links out of range, sibling trees that visit a node twice, and counts beyond sane limits. None of this costs much, and a crafted `.msi` otherwise hangs or crashes the reader.

7 The MSI database inside the compound file

In this chapter

7.1	Streams	39
	Stream names	39
7.2	The string pool: <code>_StringPool</code> and <code>_StringData</code>	39
	Code page	40
7.3	System tables	40
	Column type bits (<code>_Columns.Type</code>)	40
7.4	Table streams	41
7.5	IDT archive files (what <code>MsiDatabaseExport</code> writes)	41
7.6	Reading safely	42

Microsoft documents the *meaning* of the Windows Installer tables (Microsoft Learn, "Database Tables"), but not how a database is laid out in the compound file. Everything on this page about the layout was established by creating databases through `msi.dll`, reading the streams back byte for byte, and checking that databases written from these rules open, export and install. [observed] unless marked otherwise.

7.1 Streams

Every table is one stream directly under the root storage. There are no sub-storages in an ordinary package.

Stream names

A table named `T` is stored under the name `U+4840` followed by `T` **packed** two characters per UTF-16 unit:

1. Map each character of this 64-character alphabet to 0..63: 0-9 -> 0..9, A-Z -> 10..35, a-z -> 36..61, . -> 62, _ -> 63.
2. Two alphabet characters in a row become one unit $0x3800 + \text{first} + (\text{second} \ll 6)$.
3. A single alphabet character (the last one, or one followed by a character outside the alphabet) becomes $0x4800 + \text{value}$.
4. Any other character is written as itself.

The packed name (with the `U+4840` prefix for tables) must fit the 31-unit CFB name limit, which allows table names of up to 60 alphabet characters. Unpacking reverses the steps.

Example: `_Columns` -> `U+4840 U+3B3F U+43F2 U+4438 U+45B1`.

Other streams:

Stream	Name
a Binary/Icon cell (OBJECT column)	<Table> . <key> packed the same way without the <code>U+4840</code> prefix, e.g. <code>Binary . Logo</code> (compound keys joined with .)
an embedded cabinet	the name after # in <code>Media.Cabinet</code> (e.g. <code>cab1.cab</code>), packed, no prefix
summary information	<code>\x05SummaryInformation</code> - not packed (see Summary information)

7.2 The string pool: `_StringPool` and `_StringData`

All strings of all tables live once in a shared pool; table cells hold string ids.

`_StringPool:`

```

u32  header      low 31 bits: the database code page; bit 31: string references are 3 bytes
then per string id 1, 2, 3, ... (id 0 is "no string" and is not stored):
u16  length      byte length in _StringData
u16  refs        bits 0-14: reference count; bit 15: set exactly when the string has a byte >=
0x80

```

- **Long strings** (64 KiB of bytes or more): the entry is length = 0, refs as above (non-zero), followed by an extra u32 with the full byte length. The pair still describes one id. Seen: a 70002-byte string used once -> 0000 8001 00011172.
- An entry with length = 0 and refs = 0 is a free id.
- **Reference count** = the number of cells that hold the string, plus, per table, 1 for its `_Tables` row and, per column, 1 for the table name in `_Columns` and 1 for the column name. (Checked by recounting every database `msi.dll` produced.)
- `_StringData` is all the strings' bytes concatenated in id order, in the database code page. Lengths are **bytes**, not characters.
- **An empty string is never stored:** writing "" into a cell stores null.
- With more than 65535 distinct strings the header's bit 31 is set and every string reference in every table becomes 3 bytes (low 16 bits, then the high byte).

Code page

The database code page is the pool header's low bits: 0 when never set, otherwise what was set through the `_ForceCodepage` import. **65001 (UTF-8) works for every table** on current Windows: Korean, Japanese and supplementary-plane (e.g. U+20000, U+1F600) file and folder names, registry keys and values, shortcut names and the product name all install with exactly the right UTF-16 text, and dialogs display them - also under a system locale whose ANSI code page cannot hold Korean (en-US, 1252), so no separate Unicode setup program is needed. A UTF-16 code page (1200) is not possible here: the pool holds single-byte-unit strings. [observed] Windows Installer does not normalize names: a folder named with decomposed Hangul (U+1100 U+1161) is installed with exactly those code units, next to - not instead of - one named U+AC00. [observed]

7.3 System tables

Table	Columns	Key
<code>_Tables</code>	Name (string)	Name
<code>_Columns</code>	Table (string), Number (i2, 1-based), Name (string), Type (i2)	Table, Number

`_Tables` does not list itself or `_Columns`. A table with no rows has no stream at all, only its `_Tables` and `_Columns` rows. `_Validation`, when present, is an ordinary table.

Column type bits (`_Columns.Type`)

Bits	Meaning
0x00FF	width: maximum string length (0 = unlimited), or 2 / 4 for integers
0x0100	valid
0x0200	localizable
0x0400	non-binary (set for text, clear for OBJECT)
0x0800	string (set for text and OBJECT)
0x1000	nullable
0x2000	part of the primary key

Values `msi.dll` uses for common SQL column types:

SQL	Type
CHAR(72) NOT NULL (key)	0x2D48
CHAR(72) (nullable)	0x1D48
CHAR(255) NOT NULL LOCALIZABLE	0x0FFF
LONGCHAR NOT NULL LOCALIZABLE	0x0F00
SHORT NOT NULL	0x0502
SHORT (nullable)	0x1502
SHORT NOT NULL (key)	0x2502
LONG NOT NULL	0x0104
OBJECT NOT NULL	0x0900

Integer columns do not have the non-binary bit.

7.4 Table streams

- A table stream is **column-major**: every row's cell of column 1, then every row's cell of column 2, and so on. The row count is the stream size divided by the row width.
- Cell widths: string reference 2 bytes (3 with long references), 16-bit integer 2 bytes, 32-bit integer 4 bytes, OBJECT 2 bytes.
- Integers are stored with the sign bit flipped: $\text{stored} = \text{value} \wedge 0x8000$ (16-bit) or $\text{value} \wedge 0x80000000$ (32-bit). **Stored 0 means null**. Consequently -32768 and -2147483648 cannot be stored.
- An OBJECT cell stores 1 when a stream exists, 0 for null.
- **Rows are sorted by the primary-key cells' stored numbers** - string ids, flipped integers - key columns in order. Windows Installer relies on this order.

A writer that assigns string ids in byte order of the strings gets rows sorted by string value for free, which is also the order an IDT export uses.

7.5 IDT archive files (what `MsiDatabaseExport` writes)

Useful as a reference output: a reader that exports byte-identical IDT files decodes the database the way Windows does.

```

line 1: column names separated by TAB
line 2: column types: s/S (string), l/L (localizable string), i/I (integer), v/V (binary),
        upper case = nullable, followed by the width (0 = unlimited; 2 or 4 for integers)
line 3: [<code page> TAB] <table name> TAB <key column> [TAB <key column> ...]
rows   : cells separated by TAB, lines end with CR LF

```

- The code page prefix on line 3 appears **only when the table holds a non-ASCII string**.
- Rows are sorted by primary-key **values** (strings by bytes, integers signed) - not by stored order.
- Null prints as an empty field. Binary cells print `<key>.ibd`; the bytes go to `<Table>/<key>.ibd`.
- TAB, CR and LF inside values are written as `0x10`, `0x11` and `0x19`. [spec]
- `_ForceCodepage.idt` is two empty lines then `<code page> TAB _ForceCodepage`.
- `_SummaryInformation.idt` has header lines `PropertyId TAB Value, i2 TAB 1255, _SummaryInformation TAB PropertyId` and one row per property. Dates are printed `yyyy/mm/dd hh:mm:ss` in the **local time of the machine doing the export**.

7.6 Reading safely

Check that the pool lengths add up to exactly the size of `_StringData`, that every string reference in a table points to a used id, that table stream sizes are a multiple of the row width, and that `_Columns` numbers each table's columns 1..n without gaps.

8 Summary information

In this chapter

8.1	Layout [spec] [observed]	43
8.2	Properties an installer package sets	43
8.3	The code page trap - keep summary strings ASCII [observed]	44

Every package has a stream `\x05SummaryInformation` (the first character is U+0005; the name is not packed). It is an OLE property set as specified in [MS-OLEPS]; Microsoft Learn ("Summary Information Stream Property Set") says which properties an installer package uses.

8.1 Layout [spec] [observed]

```

u16  byte order      FFFE
u16  version         0 (msi.dll writes 0)
u32  OS field        msi.dll writes 0x00020206
16   CLSID           zero
u32  section count   1
16   FMTID           F29F85E0-4FF9-1068-AB91-08002B27B3D9, stored E0 85 9F F2 F9 4F 68 10 AB 91
08 00 2B 27 B3 D9
u32  section offset  48

section:
u32  section size
u32  property count
count x (u32 property id, u32 offset from the section start)
values, each padded to a multiple of 4 bytes:
VT_I2 (2)           u32 type, i16 value, 2 bytes padding
VT_I4 (3)           u32 type, i32 value
VT_LPSTR (30)       u32 type, u32 byte count including the NUL, bytes, NUL, padding
VT_FILETIME (64)    u32 type, u64 100-ns intervals since 1601-01-01 UTC

```

8.2 Properties an installer package sets

Id	Name	Type	Value
2	Title	LPSTR	e.g. Installation Database
3	Subject	LPSTR	product name
4	Author	LPSTR	manufacturer
5	Keywords	LPSTR	e.g. Installer
7	Template	LPSTR	<platform>;<language id>: x64;1033, Arm64;1042, Intel;1033
9	Revision Number	LPSTR	the package code : a GUID, new for every different package file
12, 13	Create / Last Save time	FILETIME	optional
14	Page Count	I4	minimum installer version: 200 for x64 and x86, 500 for Arm64
15	Word Count	I4	bit 1 = files are compressed (in cabinets); 2 for a normal package
18	Creating Application	LPSTR	any

8.3 The code page trap - keep summary strings ASCII [observed]

Property 1 (CodePage, VT_I2) says how the LPSTR bytes are encoded. On Windows 11 25H2:

- **65001:** `msi.dll` writes the strings correctly as UTF-8, but reads the code page back as the signed value -535, fails to convert every string (the installation log says "Failed to retrieve # 9 summary info property. GetLastError returned 87, the code page is -535") and refuses the package with error 1620.
- **1200 (UTF-16):** refused the same way (log: "... the code page is 1200"), even with complete UTF-16 strings and byte counts that follow [MS-OLEPS].
- **No code page property and ASCII strings:** installs.

So: leave property 1 out and write every summary string in ASCII. Localized product names belong in the Property table (ProductName), which uses the database code page - and 65001 works there (see The MSI database inside the compound file).

9 The smallest package that installs, repairs, upgrades and uninstalls

In this chapter

9.1	Tables and columns	45
9.2	Property	46
9.3	Directory	46
9.4	Components, features, files	46
9.5	Empty folders	47
9.6	Media and the cabinet	47
9.7	Major upgrade and downgrade refusal	47
9.8	Running an installed program to register and unregister	47
9.9	Registry values	48
9.10	Shortcuts	48
9.11	Removing files and duplicating them	48
9.12	Environment variables	49
9.13	INI files	49
9.14	Searches and launch conditions	49
9.15	Services, fonts, permissions	50
9.16	Cabinets, administrative images, advertisement	50
9.17	Per-user and dual packages	50
9.18	Files in use	50
9.19	Sequences	50
9.20	Dialogs	51
	Choices during installation	52
	Several languages in one package	53
9.21	What this recipe does not cover yet	53

Table meanings are documented on Microsoft Learn ("Database Tables", "Standard Actions", "Suggested InstallExecuteSequence"). This page is a tested recipe: a package built exactly like this installs per machine, survives repair, upgrades an older version, refuses a downgrade and uninstalls cleanly on Windows 11. [observed]

9.1 Tables and columns

Types use the notation of The MSI database inside the compound file (s72 = CHAR(72), upper case = nullable, l = localizable, i2/i4 integers). Keys are in bold.

Table	Columns
Property	Property s72, Value l0
Directory	Directory s72, Directory_Parent S72, DefaultDir l255
Component	Component s72, ComponentId S38, Directory_ s72, Attributes i2, Condition S255, KeyPath S72
Feature	Feature s38, Feature_Parent S38, Title L64, Description L255, Display I2, Level i2, Directory_ S72, Attributes i2
FeatureComponent	Feature_ s38, Component_ s72

Table	Columns
File	File s72, Component_ s72, FileName l255, FileSize i4, Version S72, Language S20, Attributes I2, Sequence i4
MsiFileHash	File_ s72, Options i2, HashPart1..HashPart4 i4
CreateFolder	Directory_ s72, Component_ s72
Media	DiskId i2, LastSequence i4, DiskPrompt L64, Cabinet S255, VolumeLabel S32, Source S72
Upgrade	UpgradeCode s38, VersionMin S20, VersionMax S20, Language S255, Attributes i4, Remove S255, ActionProperty s72
CustomAction	Action s72, Type i2, Source S72, Target S255
InstallExecuteSequence	Action s72, Condition S255, Sequence I2
InstallUISequence	

`File.Sequence` and `Media.LastSequence` as 32-bit integers let a package hold more than 32767 files.

9.2 Property

`ProductCode`, `ProductName`, `ProductVersion` (a.b.c, a and b up to 255, c up to 65535; a fourth part is allowed but ignored when versions are compared), `Manufacturer`, `ProductLanguage` (1033, 1042, ...), `UpgradeCode`, `ALLUSERS=1` (per machine), `REBOOT=ReallySuppress`, `MSIRESTARTMANAGERCONTROL=Disable` (see "Files in use" below), and `SecureCustomProperties` listing the two upgrade properties below.

9.3 Directory

- `TARGETDIR` with no parent and `DefaultDir = SourceDir`.
- Each standard folder used (`ProgramFiles64Folder`, `ProgramFilesFolder`, ...) as a child of `TARGETDIR` with `DefaultDir = ..` Windows resolves these names itself. For an x64 or Arm64 package use the ...64Folder names, for x86 the plain ones.
- Your folders below them with `DefaultDir = SHORT|Long name`. The long name may be any Unicode text (with code page 65001). The short name must be a valid 8.3 ASCII name, unique among all files and folders of the same parent; when the long name already is a valid 8.3 name it can be written alone.

9.4 Components, features, files

- **One component per file**, the file as its key path. `ComponentId` is a GUID that must stay the same for the same resource at the same place for the life of the product (see Deterministic identities and reproducible packages). `Attributes` 256 (64-bit) for x64 and Arm64 packages, 0 for x86.
- A feature with `Level 1`; `Display 0` hides it. Map every component in `FeatureComponents`. A child feature names its parent in `Feature_Parent`. A feature whose `Level` is above `INSTALLLEVEL` (1 unless the package sets it) is not installed by default; `MsiConfigureFeature` can add it later, and its files then arrive. [observed]
- `File.FileName` is `SHORT|Long` like `DefaultDir`. `Attributes` 512 marks the file vital. Sequence numbers 1..n are the order of the files inside the cabinet.
- Files without a version resource need an **MsiFileHash** row: the MD5 of the file, read as four little-endian 32-bit integers (stored as signed values). Installed files hash with `MsiGetFileHash` to exactly these values. Files with a version resource put the version in `File.Version` instead (and need no hash).

9.5 Empty folders

A folder that must exist without files gets its own Directory row, a component whose Directory_ is that folder and whose KeyPath is null (the folder itself is the key path), a CreateFolder row pointing at both, and a FeatureComponents row. CreateFolders makes it at install time; RemoveFolders removes it at uninstall if it is empty. [observed] With component attribute 16 (permanent) the folder stays after uninstall while everything else is removed. [observed]

9.6 Media and the cabinet

One row: DiskId 1, LastSequence n, Cabinet #cab1.cab - the # means "a stream inside this package named cab1.cab". The cabinet holds the files named by their File keys, in Sequence order (Cabinets, MSZIP and deflate). Summary Word Count 2 says the files are compressed into cabinets.

9.7 Major upgrade and downgrade refusal

Two Upgrade rows with the package's UpgradeCode:

VersionMin	VersionMax	Attributes	ActionProperty	Meaning
this version	-	0x102 (only detect, min inclusive)	NEWER_FOUND	same or newer version installed
-	this version	0x001 (migrate features)	OLDER_FOUND	older version installed: remove it

The Upgrade table matches by UpgradeCode, version and language only - not by architecture. If an x64 and an x86 build share one UpgradeCode, installing one removes the other as "older". Give each architecture its own UpgradeCode when both may be installed side by side (verified: two architectures with separate codes install and uninstall independently). [observed]

An error custom action (type 19, Target = the message; it is a formatted string, so [ProductName] works) conditioned on NEWER_FOUND stops the installation with error 1603 when a same-or-newer version is present. Every version needs a new ProductCode (and every package file a new package code).

9.8 Running an installed program to register and unregister

Some products must register themselves (an input method, a shell extension) through their own program rather than through table rows. The pattern that keeps install, repair, upgrade and removal atomic uses the component action state of that program's component C (\$C = what this installation does to it, ?C = its state before; 2 absent, 3 local) [spec], with type 18 actions (Source = the program's File key, Target = its arguments), all deferred (0x400) and not impersonated (0x800):

Action	Type	Where	Condition	Runs
...UndoRollback	18+0x100+0x400+0x800+0x400	before RemoveFiles	\$C=2 AND ?C=3	register (rollback of the next row)
...Undo	18+0x400+0x800	after it, before RemoveFiles	\$C=2 AND ?C=3	unregister
...DoRollback	18+0x100+0x400+0x800+0x400	after InstallFiles	\$C>2 AND ?C<>3	unregister
...RedoRollback	same	after it	\$C>2 AND ?C=3	register
...Do	18+0x400+0x800	after both	\$C>2	register

A rollback action must be sequenced *before* the action it undoes: rollback runs the script backwards, and only rollback actions already in the script are run. Rollback actions ignore their exit code (0x40); forward actions do not, so a failing register or unregister fails the installation and rolls it back. Observed on Windows 11: repair runs ...Do again; a failing unregister rolls the removal back and

the program (still present) registers again; when an upgrade fails, the new package's ...DoRollback runs, then the old version's files return and the old package's own ...UndoRollback registers the old program again. [observed]

Nothing that writes to the script may stand between InstallInitialize and RemoveExistingProducts: with any deferred or rollback custom action there, every upgrade stops with error 2613 ("RemoveExistingProducts action sequenced incorrectly"). [observed]

9.9 Registry values

Registry (**Registry** s72, Root i2, Key l255, Name L255, Value L0, Component_ s72) is written by WriteRegistryValues (5000) and undone at uninstall; RemoveRegistry (same columns without Value) by RemoveRegistryValues (2600). Root 0 = HKCR, 1 = HKCU, 2 = HKLM. The value's first characters choose its type: #x binary (hex), #% expandable string, # followed by digits a DWORD (up to #4294967295), [~] anywhere a multi-string ([~]a[~]b[~]); a plain string that starts with # is written as ##. A Name of - in RemoveRegistry deletes the whole key. A failed installation restores overwritten values and removed keys. [observed]

A component whose key path is a registry value has attribute 4 and KeyPath = the Registry row. The component's 64-bit attribute (256) chooses the registry view: without it, a 64-bit package writes to the 32-bit view (WOW6432Node). A 32-bit component in a 64-bit package is legal; a 64-bit component in a 32-bit package is not.

REG_QWORD cannot be expressed in the Registry table. rubrapack writes it with a DLL custom action from the Binary table (type 1): an immediate action reads the component action states and the current values and passes two lists as CustomActionData - one to a deferred action that writes or deletes, one to its rollback twin that restores what was there. The DLL's bitness must match the package (the engine loads it into a custom action server of the package's architecture). [observed: x64 and x86]

File types and URL schemes are plain Registry rows under Root 0 (HKCR) in the program's component, not the Extension/Verb/ProgId tables (those bring advertisement and repair on first use): .ext (default) = the ProgId; ProgId (default) = its description, ProgId\DefaultIcon = [#File], 0, ProgId\shell\open\command = "[#File]" "%1"; for a scheme, scheme (default) = URL:<description>, URL Protocol = empty (a Null Value with a Name writes an empty string), and the same DefaultIcon and shell\open\command. Root 0 follows the installation: HKLM\Software\Classes per machine, HKCU\Software\Classes per user. Opening a file of a type only this program claims then starts it directly. [observed]

9.10 Shortcuts

Shortcut (**Shortcut** s72, Directory_ s72, Name l128 SHORT|Long without .lnk, Component_ s72, Target [#FileKey] for a non-advertised shortcut, Arguments (formatted), Description (plain text), Hotkey, Icon_, IconIndex, ShowCmd, WkDir = a Directory key) is written by CreateShortcuts (4500) and removed by RemoveShortcuts (3200). Put the shortcut in its target file's component. A folder made only for shortcuts is not removed by itself: add a RemoveFile row (FileName null, DirProperty = the folder, InstallMode 2) for it and each parent below the standard folder. With ALLUSERS=1, ProgramMenuFolder and DesktopFolder are the all-users Start menu and the Public Desktop. A failed installation leaves no shortcut or folder; repair recreates a deleted one. [observed]

9.11 Removing files and duplicating them

RemoveFile (**FileKey** s72, Component_ s72, FileName L255 with */? or null for the folder, DirProperty s72, InstallMode i2: 1 install, 2 uninstall, 3 both) runs in RemoveFiles when its component is installed (1) or removed (2). A component with no key path file and a Directory_ works as the

carrier.DuplicateFile (**FileKey** s72, Component_ s72 = the source's component, File_ s72, DestName L255 SHORT|Long, DestFolder S72) runs in DuplicateFiles (4210) and RemoveDuplicateFiles. Files removed at install come back when the installation fails; a folder that existed before the installation is left in place at uninstall, even when empty. [observed]

9.12 Environment variables

Environment (**Environment** s72, Name l255, Value L255 formatted, Component_ s72), written by WriteEnvironmentStrings (5200) and RemoveEnvironmentStrings. Name prefixes: * system variable (without it a per-user one), = create or set, - undo when the component is removed. Value [~];x appends ;x, x;[~] prepends. With -, uninstall deletes a set variable and takes only the appended or prepended part out of a variable that existed before; without - nothing is undone. A failed installation restores the previous values. [observed]

9.13 INI files

IniFile (**IniFile** s72, FileName l255 SHORT|Long, DirProperty S72, Section l96, Key l128, Value l255 formatted, Action i2: 0 add line, 1 create line, 3 add tag, Component_ s72) is applied by WriteIniValues (5100) and undone when the component is removed; RemoveIniFile (same columns, Value nullable, Action 2 remove line, 4 remove tag) by RemoveIniValues, which runs before InstallFiles. An add tag appends ,value to a comma list and uninstall removes just that item. Non-ASCII file, section, key and value names work. A failed installation restores the file. [observed]

9.14 Searches and launch conditions

AppSearch (**Property, Signature_**) at 50 in both sequences sets the property from a locator with the same signature: RegLocator (Root, Key, Name, Type 2 = raw value, +16 = 64-bit view), DrLocator (Path may start with a folder property, like [System64Folder]) with a Signature row for a file (FileName, MinVersion), or CompLocator (ComponentId, Type 1 = key file). The property must be public and listed in SecureCustomProperties to reach the server side.

A search can give a directory its default: a directory's property set before CostFinalize is where the directory resolves. rubrapack searches into a property of its own (RpFound_<ID>) and copies it with a type-51 action (Source = the directory, Target = [RpFound_<ID>]) at 51 in both sequences under the condition RpFound_<ID> AND NOT <DIR>, so a directory given on the command line is left alone. The registry locator then has Type 0 (a folder, which must exist; the value comes back with a trailing backslash) instead of 2: **AppSearch looks a folder-type RegLocator up in the Signature table, which must exist, even empty** - without it the installation stops with 2228. [observed: a /qn major upgrade installs into the folder the earlier version recorded; DIR= on the command line wins]

rubrapack's install folder guard (guard = true) is an immediate DLL custom action (type 1, the helper DLL in Binary) at 1010 in InstallExecuteSequence, after CostFinalize has resolved the directories and before InstallValidate, under NOT Installed. It reads the Directory keys from a property, gets each path with MsiGetTargetPath, and refuses a path in which an existing part is a reparse point or not a folder, or whose folder exists with an owner (GetNamedSecurityInfo) other than S-1-5-18, S-1-5-32-544 or TrustedInstaller; it then shows the message with MsiProcessMessage(INSTALLMESSAGE_ERROR) and returns ERROR_INSTALL_FAILURE, so the installation ends with 1603 before any file is written. [observed]

LaunchCondition (**Condition**, Description formatted) at 100 in both sequences stops the installation with the description when a condition is false. [observed]

9.15 Services, fonts, permissions

ServiceInstall (Name, DisplayName and Description formatted, ServiceType 0x10, StartType 2/3/4, ErrorControl 1, StartName null or NT AUTHORITY\LocalService, Arguments formatted, Component_ = the exe's component) with InstallServices (5800); ServiceControl Event flags 0x1 start on install, 0x2 stop on install, 0x20 stop on removal, 0x80 delete on removal, Wait 1, with StopServices (1900), DeleteServices (2000), StartServices (5900). Font (File_, FontTitle null = read from the font) with RegisterFonts/UnregisterFonts; the file must be in FontsFolder itself. MsiLockPermissionsEx (**MsiLockPermissionsEx**, LockObject, Table = CreateFolder, File, Registry or ServiceInstall, **SDDLText** (the column name), Condition) needs Windows Installer 5.0 (summary page count 500); a folder gets it through a CreateFolder row. A failed installation leaves no service or font. [observed]

9.16 Cabinets, administrative images, advertisement

Several cabinets: one Media row each (DiskId 1..n, LastSequence = the last file's sequence in it); embedded ones are streams named in Cabinet with #, external ones are files next to the package named in Cabinet without # (long names work). AdminExecuteSequence (CostInitialize 800, FileCost 900, CostFinalize 1000, InstallValidate 1400, InstallInitialize 1500, InstallAdminPackage 3900, InstallFiles 4000, InstallFinalize 6600) and AdminUISequence make msiexec /a write an uncompressed image that installs like the original; AdvtExecuteSequence (CostInitialize, CostFinalize, InstallValidate, InstallInitialize, PublishFeatures 6300, PublishProduct 6400, InstallFinalize) makes msiexec /jm advertise the product. [observed]

9.17 Per-user and dual packages

The single-package form: ALLUSERS=2 with MSIINSTALLPERUSER=1 installs per user - the engine points ProgramFilesFolder/ProgramFiles64Folder at %LOCALAPPDATA%\Programs and the menu and desktop folders at the user's - and ALLUSERS=1 MSIINSTALLPERUSER="" installs per machine. Summary Word Count bit 8 says no elevation is needed. Registry Root -1 (HKMU) is HKCU per user and HKLM per machine; an Environment name without * is a user variable. Once a per-user product is installed the engine deletes MSIINSTALLPERUSER, so a launch condition that tests it must read Installed OR ... or the product can no longer be removed; in general, write every launch condition as Installed OR (...). [observed]

9.18 Files in use

When a file to be replaced or removed is held by a running program (a DLL loaded into it):

- With Restart Manager, the default, Windows Installer tries to **shut down every program holding such a file, even in a silent installation** (/qn), and restarts it afterwards; if one does not close, the whole installation fails (1601). [observed]
- With MSIRESTARTMANAGERCONTROL=Disable in the Property table, the engine moves the held file aside (C:\Config.Msi*.rbf), puts the new file in place at once and returns 0; the program keeps running on the old copy, programs started afterwards load the new file, and the old copy is deleted at the next restart. [observed]
- A property given on the command line reaches only the package being installed. During an upgrade, the removal of the old version follows **the old package's** Property table - author the property from the first version on. [observed]

9.19 Sequences

InstallExecuteSequence (conditions in brackets):

```
FindRelatedProducts 25, <refuse-downgrade action> 30 [NEWER_FOUND], CostInitialize 800,
FileCost 900, CostFinalize 1000, MigrateFeatureStates 1200, InstallValidate 1400,
```

```
InstallInitialize 1500, RemoveExistingProducts 1501, ProcessComponents 1600,
UnpublishFeatures 1800, RemoveFiles 3500, RemoveFolders 3600, CreateFolders 3700,
InstallFiles 4000, RegisterUser 6000, RegisterProduct 6100, PublishFeatures 6300,
PublishProduct 6400, InstallFinalize 6600
```

InstallUISequence: FindRelatedProducts 25, the refuse-downgrade action 30 [NEWER_FOUND], CostInitialize 800, FileCost 900, CostFinalize 1000, MigrateFeatureStates 1200, ExecuteAction 1300 (and the dialogs, see Dialogs).

RemoveExistingProducts right after InstallInitialize puts the removal of the old version inside the new installation's transaction, so a failed upgrade rolls back to the old version. [observed: a deferred custom action that fails right after InstallFiles makes the upgrade end with 1603, and the old version is registered again with its files byte for byte; a failed first installation leaves no files and no registration.]

9.20 Dialogs

A package with dialogs carries these tables (types as above):

Table	Columns
Dialog	Dialog s72, HCentering i2, VCentering i2, Width i2, Height i2, Attributes I4, Title L128, Control_First s50, Control_Default S50, Control_Cancel S50
Control	Dialog s72, Control s50, Type s20, X i2, Y i2, Width i2, Height i2, Attributes I4, Property S72, Text L0, Control_Next S50, Help L50
ControlEvent	Dialog s72, Control s50, Event s50, Argument s255, Condition S255, Ordering I2
ControlCondition	Dialog s72, Control s50, Action s50, Condition s255
EventMapping	Dialog s72, Control s50, Event s50, Attribute s50
TextStyle	TextStyle s72, FaceName s32, Size i2, Color I4, StyleBits I2
UIText	Key s72, Text L255
Binary	Name s72, Data v0 (a stream)
RadioButton	Property s72, Order i2, Value s64, X i2, Y i2, Width i2, Height i2, Text L0, Help L50
ComboBox	Property s72, Order i2, Value s64, Text L64

What the engine checks when it shows a dialog (each seen as an error dialog carrying the number):

- **Tab order.** Control_Next must form one cycle through every control that has one, starting at Control_First. A chain that leaves a control out, or ends at a control whose Control_Next is empty, stops the installation with 2810 or 2809. Static text takes no part in the cycle. [observed] rubrapack checks every dialog it builds and refuses to write a broken one (RP0010).
- **The error dialog.** The dialog named by the ErrorDialog property needs a Text control named ErrorText and an Icon control named ErrorIcon (2835 otherwise); its Attributes carry the error-dialog bit (0x10000), and push buttons named A, C, I, N, O, R, Y with EndDialog arguments ErrorAbort .. ErrorYes are shown as the message needs. [observed]
- **Brackets in texts.** Text is a formatted string: [Next] is read as a property and shows as nothing. [observed] Quote button names in running text instead.
- **Files in use.** With Restart Manager off, the engine shows the dialog named FilesInUse (buttons ending with Retry, Ignore, Exit) with a ListBox bound to FileInUseProcess. The **ListBox table must exist, even empty**: without it the engine logs 2205 and skips the dialog, and the installation goes on as if Ignore was chosen. In a major upgrade the dialog comes twice (the new files, then the old version's removal). After Ignore the held file is moved aside and the installation ends with 0. [observed]

- **Error icon.** The engine puts its own icon for the message type into `ErrorIcon` (a warning triangle for a launch condition), whatever the control's Binary holds. [observed]
- **List order.** A `ComboBox` lists its items alphabetically unless the control has the `Sorted` attribute (0x10000); with it they follow the `Order` column. [observed] rubrapack sets it.
- **Elevation.** A per-machine installation started interactively by a non-elevated user asks for consent (UAC) when the execute sequence starts, on the secure desktop. [observed]
- **Code page.** With the database code page 65001, Korean titles, texts and RTF license text show correctly under a Korean and under an English (1252) system locale. observed, see [msi-database.md]

rubrapack's sets: every dialog is 370 x 270 dialog units with a banner strip (a `Bitmap` control 0,0,370,44 whose picture is `Binary.RpBanner`), the title in bold (`{\RpTitle}`, a `TextStyle`) and a line of description in it, a `Line` at 44 and at 234, and `Back / Next / Cancel` at y 243. The `InstallUISequence` adds the welcome (1230, `NOT Installed`) or the maintenance dialog (1240, `Installed AND NOT RESUME AND NOT Preselected`), the progress dialog (1280, modeless, subscribed to `ActionText` and `SetProgress` through `EventMapping`) and the three exit dialogs at -1 (success), -2 (cancelled) and -3 (fatal). The install folder dialog puts the folder's `Directory` key in a `PathEdit`, and its `Next` runs `SetTargetPath` before `NewDialog`, which checks the path. The license text is a `ScrollableText` control holding RTF: plain text becomes `\uN?` escapes (characters above U+FFFF as a surrogate pair), one `\par` per line.

Author pages are ordinary `Dialog` rows in the same frame; `Control_Next` runs through their controls in position order and on to `Back`, `Next` and `Cancel`. A radio group is a `RadioButtonGroup` control whose buttons are `RadioButton` rows (positions relative to the group); a drop-down list is a `ComboBox` control with `ComboList` (0x20000) and `Sorted`, filled from `ComboBox` rows. Their properties are added to `SecureCustomProperties`, so values chosen in the dialogs or given on the command line reach the execute sequence. [observed]

Choices during installation

What rubrapack writes for the choices a user makes [observed, Windows 11 26100]:

- **Features.** `Feature.Attributes` 0x10 (`UIDisallowAbsent`) keeps "will be unavailable" out of the tree's menu; 0x02 (`FollowParent`) installs a feature where its parent is. A feature that is off unless a condition holds is a `Condition` row (**Feature**_s38, **Level** i2, `Condition` S255) with `Level` 0 and the condition `NOT Installed AND NOT (<when>)`. **Without NOT Installed the row turns the feature off again at removal, when the property is gone, and its files are left behind.**
- **Components.** when on a file, a registry value, an environment variable or an INI value is the component's `Condition`, evaluated when the component is first installed (no `Transitive` attribute: a repair keeps what is there). A shortcut lives in its target file's component; one with `when` gets a component of its own in the shortcut's folder, whose key path is a registry value under root -1 (`HKMU`: `HKLM` or `HKCU` as installed), with the attribute 0x4.
- **Icons.** An `Icon` row (**Name** s72 ending in `.ico`, `Data` v0) holds the `.ico`; `ARPPRODUCTICON` names it for `Installed` apps, which the engine registers as the product's `ProductIcon` under `HKLM\SOFTWARE\Classes\Installer\Products` (not as `DisplayIcon`), and `Shortcut.Icon_ / IconIndex` 0 give a shortcut its own.
- **Keep.** A file left at removal is a component with 0x10 (`Permanent`).
- **Launch.** A type-34 custom action with 0xC0 (asynchronous, no wait): `Source` = the program's folder, `Target` = "[#FileKey]" args. The finished dialog's `Finish` runs it through `DoAction` before `EndDialog` under `RPLAUNCH` = "1" `AND NOT Installed`; `Installed` still has its value from the start of the setup, so repair and removal do not start it. It runs in the setup's client process, with the rights of the user who started the setup.
- **Change.** The maintenance dialog's `Change` is a `NewDialog` to the dialog with the `SelectionTree`; that dialog's `Back` goes to the maintenance dialog under `Installed` and to the install folder dialog under `NOT Installed`; the ready dialog's `EndDialog Return` then applies the new feature states.

- **Scope.** In a dual package (ALLUSERS=2, MSIINSTALLPERUSER=1 by default) the folder properties follow the per-user default when the dialogs start. The scope page's Next sets [ALLUSERS] and [MSIINSTALLPERUSER] ({} clears it), then puts the install folder's path under [%ProgramW6432] (or [%ProgramFiles(x86)] for an x86 package) or [LocalAppDataFolder]Programs into the directory's property and runs SetTargetPath; the execute sequence resolves the other folders itself with the final scope.

Several languages in one package

The dialog tables hold one language at a time, but every text they show can come from a property, so one set of dialogs can speak several languages. What the engine does [observed, Windows 11 26100]:

- A Text, PushButton, CheckBox, RadioButton or ComboBox text of [RpT_X], and a Dialog Title of [P], show the property's value; each dialog reads it when it is created, so a value changed by one dialog shows on the next.
- A text style prefix at the start of the value ({} \RpTitle_ko} . . .) applies.
- **The value is not formatted again:** [ProductName] inside a property's value shows literally. The text must be formatted when it is put into the property - by a [RpT_X] ControlEvent, whose Argument is formatted when the event runs, or by a type-51 custom action, whose Target is.
- Setting [DefaultUIFont] by an event changes the face of every control without its own style on the next dialog.
- A dialog's Control_First must be visible when the dialog opens (2836 otherwise); a control hidden by ControlCondition may stay in the tab cycle.

rubrapack's layout with [ui] languages: English is language 0. For every text X and language L, RpT_X_L in the Property table holds the text with [ProductName], [Manufacturer] and [ProductVersion] put in at build time (and the title style for headings); RpT_X holds the English one. The InstallUISequence starts with type-51 actions: RPLANGUAGE = L when unset and UserLanguageID (17), then SystemLanguageID (18), is one of L's LANGIDs, else en (19); then at 21 one action per text copies [RpT_X_L] into RpT_X under RPLANGUAGE = "L", and one sets DefaultUIFont. A text that uses other properties is copied as its formatted source instead. The language page (RpLanguageDlg, 1225, a RadioButtonGroup on RPLANGUAGE) replaces the welcome and maintenance rows; its Next runs the same copies as ControlEvents, then NewDialog to the welcome or the maintenance page. A license per language is a ScrollableText per language in the same place, each with Show/Hide ControlConditions on RPLANGUAGE. The UIText table has one language: English.

9.21 What this recipe does not cover yet

Custom actions other than the error type, the register pair and the REG_QWORD helper above; and _Validation (needed by validation tools, not by the installer). These pages grow as rubrapack implements them.

10 Program files: machine type and version

In this chapter

10.1	Finding the headers [spec]	54
10.2	The version resource [spec]	54
10.3	What goes into the package [observed]	54
10.4	Reading safely	55

A package writer reads two facts from every Portable Executable (.exe, .dll, ...) it packs: the machine type, to refuse a file built for the wrong architecture, and the version resource, because Windows Installer treats versioned and unversioned files differently. Layout from the Microsoft PE/COFF specification and the VS_VERSIONINFO documentation. [spec]

10.1 Finding the headers [spec]

```
offset 0      "MZ"
offset 0x3C   u32  e_lfanew: offset of the PE signature
e_lfanew     "PE\0\0"
+4           COFF header: u16 machine, u16 number of sections, ..., u16 optional header size
(at +16)
+24          optional header: u16 magic 0x10B (PE32) or 0x20B (PE32+); the data directories
              start at +96 (PE32) or +112 (PE32+), preceded by their count
then         section table: 40 bytes per section (virtual size +8, virtual address +12,
              raw size +16, raw offset +20)
```

Machine values: 0x8664 x64, 0xAA64 Arm64, 0x014C x86. A file that starts with MZ but has no valid PE\0\0 header is an ordinary data file.

10.2 The version resource [spec]

1. Data directory 2 is the resource table (an RVA and size). Map RVAs to file offsets through the section table.
2. Resource directories are three levels deep: type -> name -> language. Each directory has a 16-byte header (named and ID entry counts at +12 and +14) followed by 8-byte entries (u32 name or ID, u32 offset; the high bit of the offset marks a sub-directory).
3. Type 16 (RT_VERSION), its first name, its first language: the leaf is a data entry (u32 RVA, u32 size).
4. The data is VS_VERSIONINFO: u16 length, u16 value length, u16 type, the UTF-16 key VS_VERSION_INFO with its NUL, padding to 4 bytes, then VS_FIXEDFILEINFO whose first u32 is 0xFEEF04BD. dwFileVersionMS (+8) and dwFileVersionLS (+12) give the version HIWORD(MS).LOWORD(MS).HIWORD(LS).LOWORD(LS).

10.3 What goes into the package [observed]

- Versioned file: File.Version = the four-part version, File.Language = the language MsiGetFileVersion reports, which is the first language of VarFileInfo\Translation (for example 1033, 1042); no MsiFileHash row. Installed files read back with the same version.
- Unversioned file: no version, a MsiFileHash row (The smallest package that installs, repairs, upgrades and uninstalls).

- Machine type: an x64 package takes x64 files, an Arm64 package Arm64 files, an x86 package x86 files. A deliberate exception (a 32-bit helper in a 64-bit package) should be declared, not silently accepted; the component's own bitness does not change.

10.4 Reading safely

Check every offset and size against the file length before using it, map RVAs only through a section that holds the whole range, and stop at the three expected directory levels - crafted files with loops or out-of-range pointers are common.

11 Cabinets, MSZIP and deflate

In this chapter

11.1	Cabinet layout [spec: MS-CAB]	56
	Block checksum [spec]	56
11.2	MSZIP [spec: MS-MCI]	56
11.3	A deflate encoder in brief [spec: RFC 1951]	57

11.1 Cabinet layout [spec: MS-CAB]

```
CFHEADER (36 bytes when no reserve fields):
  "MSCF", u32 0, u32 cabinet size, u32 0, u32 offset of the first CFFILE, u32 0,
  u8 minor version 3, u8 major version 1, u16 folder count, u16 file count, u16 flags (0),
  u16 set id, u16 cabinet index (0)
CFFOLDER (8 bytes each):
  u32 offset of the folder's first CFDATA, u16 number of CFDATA blocks, u16 compression
  (0 = none, 1 = MSZIP)
CFFILE (per file):
  u32 uncompressed size, u32 offset inside the folder's uncompressed data, u16 folder index,
  u16 date, u16 time, u16 attributes, NUL-terminated name
CFDATA (per block):
  u32 checksum, u16 compressed bytes, u16 uncompressed bytes (at most 32768), data
```

- In an MSI, file names in the cabinet are the File table keys, which are ASCII.
- DOS date (year - 1980) << 9 | month << 5 | day, time hour << 11 | minute << 5 | second / 2. A fixed date (rubrapack writes 1980-01-01 00:00) keeps output deterministic; attributes 0x20.
- One folder holds at most 65535 blocks (about 2 GiB); more data needs more folders or cabinets.

Block checksum [spec]

```
checksum(bytes, seed):
  xor every complete little-endian u32 of the bytes into seed;
  the 1-3 remaining bytes form one more u32 with the FIRST remaining byte in the HIGHEST
  position used (3 left: b0<<16 | b1<<8 | b2; 2 left: b0<<8 | b1; 1 left: b0), xor it in.
block checksum = checksum(the 4 bytes "compressed size, uncompressed size" as stored,
                           checksum(block data, 0))
```

A checksum of 0 means "not computed"; readers accept it.

11.2 MSZIP [spec: MS-MCI]

Each CFDATA block of a folder with compression type 1 holds:

```
"CK" (43 4B), then one complete raw deflate stream (RFC 1951) for this block's
uncompressed bytes (at most 32768), ending with a final block.
```

The decompressor keeps the previous blocks of the same folder as its dictionary, so an encoder *may* refer back into earlier blocks. An encoder that never does (each block compressed on its own) produces valid MSZIP too. rubrapack gives each block the previous block's 32 KiB of uncompressed data as a preset dictionary (about 9% smaller on mixed data than blocks on their own), and since that dictionary is the input, not the output, the blocks are still compressed on several threads at once, with the same bytes whatever their number. It starts a new folder at a file boundary before

65535 blocks; the dictionary does not cross folders. Windows extracts all of it: a package with twelve folders installs, repairs and removes with every file byte-identical. [observed]

11.3 A deflate encoder in brief [spec: RFC 1951]

1. **Match finding (LZ77)**: hash the next 3 bytes, walk a chain of earlier positions with the same hash within 32 KiB, keep the longest match (3..258 bytes). Longer chain walks give better compression; a "lazy" step that also tries the next position and emits a literal when that match is longer helps noticeably.
2. **Blocks**: group up to a few ten thousand symbols. For each block compute the size of three encodings and write the smallest:
 - fixed Huffman codes (literal/length 0-143: 8 bits, 144-255: 9, 256-279: 7, 280-287: 8; distances 5 bits);
 - dynamic Huffman codes: code lengths limited to 15 bits (7 for the code-length alphabet); package-merge gives optimal limited lengths; send HLIT, HDIST, HCLEN, the code-length code in the order 16 17 18 0 8 7 9 6 10 5 11 4 12 3 13 2 14 1 15, then the run-length coded lengths (16 = repeat previous 3-6, 17 = zeros 3-10, 18 = zeros 11-138);
 - stored (byte-aligned LEN, NLEN, raw bytes; at most 65535 per stored block).
3. Huffman codes are written most significant bit first; everything else least significant bit first.
4. Always give at least two symbols a code length (pair a lone symbol with a dummy), so every code is complete - some decoders reject incomplete codes.

Check an encoder by inflating its output with an independent implementation (for example the zlib library with raw deflate, window bits -15) and an MSI built with it by installing it and comparing installed files' hashes (Checking your output against Windows).

12 Deterministic identities and reproducible packages

In this chapter

12.1	Name-based GUIDs (UUID version 8) [spec: RFC 9562]	58
12.2	Keys	58
12.3	Reproducible packages	59

Windows Installer identifies things by GUIDs and keys that must be **stable**: the same component must keep the same ComponentId in every version, or repair and upgrade misbehave. Generating them from the content, instead of storing them by hand, keeps them stable and makes builds reproducible. This page describes rubrapack's rules; other rules work too as long as they are stable.

12.1 Name-based GUIDs (UUID version 8) [spec: RFC 9562]

```
digest = SHA-256( u32le(len(tag)) || tag
                  || u32le(1)                                     (format version)
                  || for each field: u32le(len(field)) || field ) (fields in UTF-8)
uuid   = digest[0..16], then byte 6 = (byte 6 & 0x0F) | 0x80 (version 8)
                  byte 8 = (byte 8 & 0x3F) | 0x80 (variant 10)
text   = "{" + hex(bytes 0-3) "-" hex(4-5) "-" hex(6-7) "-" hex(8-9) "-" hex(10-15) + "}"
        upper case, bytes in order
```

Length prefixes keep ("ab", "c") and ("a", "bc") apart; the tag keeps different kinds of identity apart.

Identity	Tag	Fields
ComponentId	rubrapack. component	upgrade code, scope (machine), component architecture, logical target path (<dir id>/<file name>), key path kind (file), key path name (the File key)
ProductCode	rubrapack. product	upgrade code, architecture, scope, the version's first three parts
package code (reproducible builds)	rubrapack. package	hex SHA-256 of the package built with an all-zero package code

The ProductCode changes with the three-part version - which is what a major upgrade needs - and the ComponentIds do not.

12.2 Keys

Table keys (File, Component, Directory) are identifiers: [A-Za-z_][A-Za-z0-9_.*]*, at most 72 characters. rubrapack uses the author's IDs for files and folders as given and derives the rest:

```
key = prefix + "_" + first 20 lower-case hex digits of SHA-256(logical name)
    prefix C for components (logical name: the File key), D for intermediate folders
    (logical name: "<standard folder>/<part>/<part>..."), F for files from wildcards
```

12.3 Reproducible packages

A package is byte-for-byte reproducible when nothing depends on the clock, the machine or the order of hash tables: fixed CFB directory times (zero), fixed cabinet dates, string ids assigned in byte order of the strings, rows sorted by key, streams in a fixed order, no random GUIDs. The one identity that must differ between package files is the package code; derive it from the content (as above) when reproducibility matters, and use a random one otherwise.

13 Authenticode signatures

In this chapter

13.1	Where the signature lives in a PE file	60
13.2	The PE digest	60
13.3	The CMS SignedData	61
13.4	MSI packages	61
13.5	Timestamps	62
13.6	ECDSA	62
13.7	What Windows reports	62

What a program has to write so that Windows accepts a signed PE file (.exe, .dll) or MSI package. Facts are tagged as in Package formats for implementers: **[spec]** for Microsoft's "Windows Authenticode Portable Executable Signature Format", the PE/COFF specification and RFC 5652 (CMS); **[observed]** for files signed by Windows' own signer (PowerShell Set-AuthenticodeSignature, which uses mssign32.dll) and checked with Get-AuthenticodeSignature / WinVerifyTrust on Windows 11.

13.1 Where the signature lives in a PE file

- The certificate table: data directory entry 4 (its entry is 8 bytes: file offset, size) of the optional header. The directory starts at optional header + 96 for PE32 (magic 0x10B) and + 112 for PE32+ (magic 0x20B); NumberOfRvaAndSizes must be at least 5. Unlike every other directory entry, its "address" is a **file offset**, not an RVA. [spec]
- At that offset, at the end of the file: WIN_CERTIFICATE { u32 dwLength, u16 wRevision = 0x0200, u16 wCertificateType = 0x0002 (PKCS_SIGNED_DATA), bCertificate[] }, where bCertificate is a DER CMS ContentInfo. dwLength includes the 8-byte header and is rounded up to 8; the extra bytes are zero. The directory entry's size equals dwLength. [spec] [observed]
- Before the table is appended, the file is padded with zeros to a multiple of 8. The padding is hashed (it is "data after the sections", below). [spec]
- The optional header's CheckSum (optional header + 64) is updated. Windows' signer adjusts the existing value rather than recomputing it: for a file whose checksum was not the PE algorithm's value, Windows keeps the difference, a fresh computation does not. CheckSum is outside the digest, so both verify. [observed]

13.2 The PE digest

In this order, with SHA-256: [spec]

1. The headers, from 0 to SizeOfHeaders, without CheckSum (4 bytes) and without the certificate table directory entry (8 bytes).
2. Every section with SizeOfRawData > 0, in increasing PointerToRawData order, SizeOfRawData bytes from PointerToRawData.
3. The data after them: from SizeOfHeaders + sum(SizeOfRawData) to the end of the file minus the certificate table.

Hashing the file linearly with the two holes gives the same result only when the sections follow each other without gaps; the steps above are the definition.

13.3 The CMS SignedData

Windows' signer writes, and accepts: [observed]

```
ContentInfo { signedData (1.2.840.113549.1.7.2), [0] SignedData {
  version 1,
  digestAlgorithms SET { sha256 + NULL },
  encapContentInfo { SPC_INDIRECT_DATA (1.3.6.1.4.1.311.2.1.4),
    [0] SpcIndirectDataContent }          -- the SEQUENCE itself, no OCTET
STRING
  certificates [0] { the signer's certificate, then intermediates; no self-signed root },
  signerInfos SET { SignerInfo {
    version 1, IssuerAndSerialNumber of the signer,
    digestAlgorithm sha256 + NULL,
    signedAttrs [0] { SpcSpOpusInfo, contentType, SpcStatementType, messageDigest },
    signatureAlgorithm rsaEncryption + NULL,
    signature (RSA PKCS#1 v1.5) } } } }
```

- `SpcIndirectDataContent ::= SEQUENCE { data, messageDigest DigestInfo }`. For a PE file data is `SEQUENCE { SPC_PE_IMAGE_DATA (1.3.6.1.4.1.311.2.1.15), SpcPeImageData }` and Windows writes `SpcPeImageData` as `30 09 03 01 00 A0 04 A2 02 80 00`: no flags, and a file link that is an empty Unicode string. `messageDigest` holds the PE digest. [observed]
- The **messageDigest attribute** is the hash of the `SpcIndirectDataContent` contents **without** its `SEQUENCE` tag and length - not of the whole element, and not of the PE file. [observed]
- The signed attributes are stored with tag `[0] (0xA0)` but the signature is computed over the same bytes with tag `SET (0x31)`, sorted as a DER SET OF. The four attributes Windows writes: `SpcSpOpusInfo (1.3.6.1.4.1.311.2.1.12) = SEQUENCE { [0] { [0] "" }, [1] { [0] "" } }, contentType = SPC_INDIRECT_DATA, SpcStatementType (1.3.6.1.4.1.311.2.1.11) = SEQUENCE { individual code signing (1.3.6.1.4.1.311.2.1.21) }, messageDigest`. No signing time. [observed]
- RSA PKCS#1 v1.5 is deterministic: a program that writes exactly this structure with the same key produces the same bytes as Windows. rubrapack does, except the CheckSum difference above. [observed]

13.4 MSI packages

Microsoft does not publish how an MSI package is hashed. The rules below were worked out by signing packages with Windows' signer and comparing: rubrapack's signature and Ex streams came out byte-identical to Windows' for the same package and key, and packages whose directory entries were given non-zero state bits, times and CLSIDs told the fields apart. [observed]

- The signature is the stream `\005DigitalSignature` at the root of the compound file: the same CMS `ContentInfo` as for a PE file, with `data = SEQUENCE { SPC_SIPINFO (1.3.6.1.4.1.311.2.1.30), SpcSipInfo }`, where Windows writes `SpcSipInfo` as `SEQUENCE { 2, OCTET STRING <the MSI SIP GUID {000C10F1-0000-0000-C000-000000000046} in its little-endian byte order>, 0, 0, 0, 0, 0 }`.
- Windows also writes `\005MsiDigitalSignatureEx`: 32 bytes, the SHA-256 of a "prehash" of the compound file's directory: The root's own times are not part of it (Windows sets the root's modification time when it saves the signed file).
 - the root entry's CLSID (16 bytes) and state bits (4 bytes, little-endian);
 - then, for every stream at the root except the two signature streams, in the order of their names compared as UTF-16LE bytes: the name (UTF-16LE, no terminating NUL), the stream size as 8 bytes little-endian, and the entry's creation and modification times as stored (8 + 8 bytes). A stream's CLSID and state bits are not part of it.

- The digest in `SpcIndirectDataContent` is the hash of: the 32-byte `MsiDigitalSignatureEx` value (when that stream is written), then every stream's contents in the same name order (again without the two signature streams), then the root CLSID.
- Only streams at the root were observed; a package with storages (embedded transforms, sub-databases) is not covered here, and rubrapack refuses to sign one.
- A signature covers what is inside the `.msi` only: cabinets outside it (Media table Cabinet values that do not start with #) are not part of the digest.

13.5 Timestamps

A signature without a timestamp stops being valid when the signing certificate expires. With an RFC 3161 timestamp, Windows checks the certificate at the stamped time instead. What `mssign32!SignerTimeStampEx2` with `SIGNER_TIMESTAMP_RFC3161` writes: [observed]

- The request's message imprint is the hash (here SHA-256) of the `SignerInfo`'s signature value: the `OCTET STRING`'s contents, without its tag and length.
- The server's `TimeStampToken` (a CMS SignedData over a `TSTInfo`, RFC 3161) goes into the `SignerInfo` as its unsigned attributes, after the signature: [1] { SEQUENCE { 1.3.6.1.4.1.311.3.3.1, SET { TimeStampToken } } }.
- Nothing else in the signature changes: the signed attributes, the signature and the outer certificates are the same bytes as before; the server's certificates travel inside the token. For an MSI package the `\005DigitalSignature` stream grows accordingly (from the mini stream to regular sectors, for a token of a few kilobytes); `\005MsiDigitalSignatureEx` and the digest do not change, since the signature streams are not part of them.
- rubrapack writes exactly this; with the same key and token its PE file is byte-identical to Windows', and so is the MSI signature stream.

`Set-AuthenticodeSignature -TimestampServer` writes an older form instead: a PKCS#9 `counterSignature` attribute (1.2.840.113549.1.9.6) and the server's certificates added to the outer certificates. Windows accepts both. [observed]

13.6 ECDSA

An ECDSA P-256 signer works for PE files, MSI packages and MSIX packages alike (Windows 11 26100 calls all three `Valid` with the root trusted). Windows' signer writes the `SignerInfo`'s `signatureAlgorithm` as `id-ecPublicKey` (1.2.840.10045.2.1) with NULL parameters - `not ecdsa-with-SHA256` - and the signature value as the DER SEQUENCE { INTEGER `r`, INTEGER `s` }. rubrapack writes the same, with deterministic nonces (RFC 6979), and reads either OID. [observed]

13.7 What Windows reports

`Get-AuthenticodeSignature` distinguishes the cases a signer needs (and names the time-stamping certificate in `TimeStamperCertificate` when there is a timestamp): `Valid` (digest, signature and chain to a trusted root), `UnknownError` with "the certificate chain ... not trusted" (digest and signature are right, the root is not trusted on this machine), `HashMismatch` (the file changed after signing), `NotSigned`. So a signature can be checked for correctness before any root is trusted. [observed]

14 Checking your output against Windows

In this chapter

14.1	The database opens and decodes	63
14.2	The package installs	63
14.3	Checks before writing	64
14.4	Microsoft's ICE rules [observed]	64
14.5	Things that look fine but are not	65

Microsoft does not publish the MSI table encoding, so the reference is Windows itself. These checks need only a Windows machine and the Windows Installer API (`msi.dll`); no SDK tools.

14.1 The database opens and decodes

1. `MsiOpenDatabase` your file read-only - the compound file and string pool are acceptable.
2. `MsiDatabaseExport` every table (and `_ForceCodepage`, `_SummaryInformation`) to IDT files.
3. Compare with what your own reader exports, or with the IDT of a reference database made through `msi.dll` (`MsiOpenDatabase` with create mode, `MsiDatabaseImport` of a `_ForceCodepage.idt` containing 65001, SQL CREATE TABLE/INSERT via views). Byte-identical IDT means your encoding matches.

14.2 The package installs

Drive the installer from a program rather than `msiexec`, so every step is synchronous and returns an error code:

Step	API	Expect
silent UI	<code>MsiSetInternalUI(INSTALLUILEVEL_NONE)</code>	
verbose log	<code>MsiEnableLog</code> (all modes)	read it when anything fails
install	<code>MsiInstallProduct(path, "")</code>	0
names	FindFirstFile on each installed path, compare the entry's own UTF-16 name exactly	equal
content	<code>MsiGetFileHash</code> of each installed file vs your <code>MsiFileHash</code> row	equal
registration	<code>MsiGetProductInfo(ProductCode, InstalledProductName)</code>	your ProductName
repair	delete one installed file, <code>MsiReinstallProduct(ProductCode, ...)</code>	file back, same name
upgrade	install version N+1 over N	old ProductCode gone, new one installed
downgrade	install version N again	refused (1603, the log names your refusing action)
features	<code>MsiQueryFeatureState</code> ; <code>MsiConfigureFeature(..., INSTALLSTATE_LOCAL)</code>	levels above <code>INSTALLLEVEL</code> absent until turned on
rollback	copy of version N+1 with a deferred custom action that fails after <code>InstallFiles</code> (type 34 + 1024, "[SystemFolder]cmd.exe" /c exit 1) and a new package code, installed over N	1603 (log: error 1722); N still registered, its files unchanged

Step	API	Expect
uninstall	MsiConfigureProduct(ProductCode, ..., INSTALLSTATE_ABSENT)	0; folders, keys and registration gone

Per-machine installs need an elevated process. Keep test products on their own UpgradeCodes and remove them in the same run.

14.3 Checks before writing

msi.dll accepts many broken databases and only fails at install time, if at all (it does not even enforce column widths). A writer should check its own tables first. rubrapack's build refuses to write a package that breaks any of these (diagnostics RP2001-RP2015):

- every cell has the column's kind; no null in a non-nullable column; strings no longer than the column width, counted in UTF-16 units (a non-BMP character counts two), and valid UTF-8; 16-bit integers within -32767..32767 and 32-bit ones not -2147483648 (those values mean null);
- no duplicate primary keys; references (Component.Directory_, File.Component_, FeatureComponents, CreateFolder, MsiFileHash.File_, Directory_Parent, Feature_Parent) point at existing rows; every sequence action is a standard action, a CustomAction or a Dialog;
- ProductCode, ProductName, ProductVersion, Manufacturer, ProductLanguage are present; GUIDs are upper case in braces, component GUIDs are unique, ProductCode differs from UpgradeCode;
- each File.Sequence is unique and covered by a Media.LastSequence;
- standard actions keep their order (costing before InstallValidate, file actions between InstallInitialize and InstallFinalize);
- the 64-bit component attribute (256) matches the summary Template platform (x64/Arm64 vs Intel);
- a file key path is a file of the same component;
- each Upgrade.ActionProperty is upper case (public) and listed in SecureCustomProperties - otherwise a per-machine install does not pass it to the server side.

14.4 Microsoft's ICE rules [observed]

The Windows SDK's "MSI Tools" include MsiVal2 and darice.cub, the Internal Consistency Evaluators (ICEs) that Microsoft's own authoring tools run: MsiVal2 package.msi darice.cub -f prints what fails. They run only on Windows, and they go further than the checks above - they know the meaning of the tables, not only their shape. What they report for rubrapack's test packages (every source of its test suite, 39 packages), and why:

- **ICE03, "Missing specifications in _Validation"** for every column: the _Validation table describes each column for validators only; Windows Installer does not read it, and rubrapack does not write it.
- **ICE82**, duplicate sequence numbers: the language actions of a package with several dialog languages share one number. They are independent set-property actions, so their order does not matter.
- **ICE34**, the language page's radio property has no Property row: on purpose, because the command line (RPLANGUAGE=ko) must win over the detection, which only runs NOT RPLANGUAGE.
- **ICE43 and ICE57**, a non-advertised shortcut in a component whose key path is a file: they assume shortcut folders are per user. In a per-machine package (ALLUSERS=1) the Start menu and desktop are the all-users folders, and a per-user or dual package installs once per user, so the "first user only" problem these ICEs guard against does not arise.
- **ICE52**, a private property in AppSearch (RpFound_<ID>, a remembered folder): private on purpose, so the command line cannot set it; AppSearch runs in both sequences, so it needs no passing to the server side.

Three findings were real and are fixed (0.4.3): the finished, cancelled and failed pages were missing from AdminUISequence, so an administrative installation (`msiexec /a`) with the full UI ended without them (ICE20); a component whose key path is its folder (an INI entry, a `[remove]`) was not listed in `CreateFolder` (ICE18); and a package with dialogs had no `ControlCondition` table, which ICE17 reads for every dialog (it stopped with error 2228 instead of checking).

14.5 Things that look fine but are not

- An `msi.dll`-made database is not a complete specification: `msi.dll` stores 65001 summary strings correctly and still cannot read them back (Summary information).
- `MsiDatabaseExport` prints summary dates in local time; compare dates in UTC.
- A successful `MsiOpenDatabase` says nothing about row order or reference counts - only installation does. Test with an install, not just an open.

15 MSIX packages

In this chapter

15.1	The ZIP archive	66
15.2	The block map (AppxBlockMap.xml)	66
15.3	[Content_Types].xml	67
15.4	The manifest (AppxManifest.xml)	67
15.5	The virtual registry (Registry.dat, User.dat)	67
15.6	The virtual file system (VFS\...)	68
15.7	Extensions	68
15.8	Bundles (.msixbundle)	69
15.9	Signatures (AppxSignature.p7x)	70
15.10	Installing an unsigned package	70

What a program has to write so that Windows reads, installs and runs an MSIX package. Facts are tagged as in Package formats for implementers: **[spec]** for Microsoft Learn (package manifest and block map schemas) and ECMA-376 Part 2 (Open Packaging Conventions); **[observed]** for packages written by Windows' own packaging API (IAppxFactory/IAppxPackageWriter in AppxPackaging.dll, part of Windows) and for rubrapack's packages read back through IAppxPackageReader and installed with Add-AppxPackage on Windows 11. rubrapack writes what is described here.

15.1 The ZIP archive

- The payload files first, then AppxManifest.xml, AppxBlockMap.xml and [Content_Types].xml, in that order. [observed]
- Every entry is ZIP64, whatever its size: the local header has version needed 4.5, flag 0x0008 (data descriptor), CRC and sizes 0 and no extra field; the data is followed by a ZIP64 data descriptor (PK\7\8, CRC-32, compressed and plain size as 8 bytes each). The central directory entry has made-by and needed 4.5 (MS-DOS), sizes and offset 0xFFFFFFFF and a ZIP64 extra field (0x0001, 24 bytes: plain size, compressed size, local header offset). The archive ends with a ZIP64 end record (size 44), its locator, and an end record whose counts and offsets are all 0xFFFF / 0xFFFFFFFF. [observed]
- Methods: stored (0) and deflate (8). AppxManifest.xml, AppxBlockMap.xml and [Content_Types].xml are deflated even when the payload is stored. [observed]
- Entry names are OPC part names: / between folders and every byte outside A-Z a-z 0-9 - . _ ~ percent-encoded, UTF-8 bytes included - data\<U+C790> %(1).txt (a Hangul syllable, a space, %, # and parentheses) is stored as data/%EC%9E%90%20%25%23%281%29.txt - and the UTF-8 name flag is not set. [Content_Types].xml keeps its brackets. [observed]
- The dates in the headers are not read; Windows writes the time of writing, rubrapack 1980-01-01 00:00 so that a package does not change from one build to the next. [observed]

15.2 The block map (AppxBlockMap.xml)

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<BlockMap xmlns="http://schemas.microsoft.com/appx/2010/blockmap"
  xmlns:b4="http://schemas.microsoft.com/appx/2021/blockmap" IgnorableNamespaces="b4"
  HashMethod="http://www.w3.org/2001/04/xmlenc#sha256">
  <File Name="data\text.txt" Size="218890" LfhSize="43">
```

```

    <Block Hash="(base64 SHA-256 of 65536 plain bytes)" Size="(compressed bytes of this
block)"/>
    ...
    <b4:FileHash Hash="(base64 SHA-256 of the whole file)"/>
  </File>
  ...
</BlockMap>

```

- One File per payload file and one for AppxManifest.xml; the block map and [Content_Types].xml are not listed. Name is the path as written, with \, not percent-encoded; Size is the plain size; LfhSize the local header's size (30 + the ZIP name's length). An empty file has no Block. [spec] [observed]
- One Block per 65536 bytes of plain data; Hash is the base64 SHA-256 of that plain block. [spec]
- A deflated file is one independent raw deflate part per block: each part decodes with a fresh decoder (nothing refers back into an earlier block) and ends with an empty stored block (00 00 FF FF), and after the last part comes an empty final block (03 00). The block's Size counts the bytes of its part, so the compressed size of the file is the sum of the Sizes plus 2 (an empty file: 2 bytes, no block). Stored files have no Size on their blocks. Incompressible data is deflated as well (into stored blocks). [observed]
- b4:FileHash appears only for files of more than one block. [observed]
- Windows' reader checks every block's hash as the file is read, and refuses a package whose file differs from its block map, both when reading and when installing. [observed]

15.3 [Content_Types].xml

One line: a Default per file extension in order of first use (lower case), xml as application/vnd.ms-appx.manifest+xml, and Override PartName="/AppxBlockMap.xml" as application/vnd.ms-appx.blockmap+xml. rubrapack adds an Override for each file without an extension, and one for /AppxManifest.xml when a payload .xml file took the xml default. [observed]

15.4 The manifest (AppxManifest.xml)

The smallest desktop application that Windows installs and starts (namespaces foundation/windows10, uap/windows10, restrictedcapabilities): [spec] [observed]

- Identity: Name (3-50 characters of A-Z a-z 0-9 . -), Publisher (the signing certificate's subject), Version (four parts, each at most 65535), ProcessorArchitecture (x64, arm64, x86).
- Properties: DisplayName, PublisherDisplayName, Logo (a PNG in the package).
- Dependencies/TargetDeviceFamily Name="Windows.Desktop" with MinVersion and MaxVersionTested.
- Resources/Resource Language.
- Application Id Executable EntryPoint="Windows.FullTrustApplication" with uap:VisualElements (DisplayName, Description, BackgroundColor, Square150x150Logo, Square44x44Logo), and the restricted capability runFullTrust.
- Paths in the manifest use \ and name files of the package.

15.5 The virtual registry (Registry.dat, User.dat)

Registry hives at the package root (the REGF format: Registry hive files (REGF)). Measured with a packaged app on Windows 11 (26100): [observed]

- Registry.dat: its REGISTRY\MACHINE\SOFTWARE key stands for HKLM\Software (the hive's root itself does not); REGISTRY\MACHINE\SOFTWARE\WOW6432Node\... is what the app sees in the 32-bit view. Microsoft's description ("registry.dat serves as the logical equivalent of HKLM\Software") is about what the app sees, not about the hive's layout.

- User.dat: its root stands for HKCU (Software\... beneath it is HKCU\Software\...).
- The app reads the package's keys merged into the real registry; nothing is written to the machine's registry, and nothing is left after removal.
- Windows' packaging tools write these hives with the Offline Registry Library (its mark "OfRg" is in the base block).

15.6 The virtual file system (VFS\...)

Files under VFS\<folder> appear to the app at the real location; the real folder does not change. Measured: ProgramFilesX64 (%ProgramFiles%), SystemX64 (System32), Common AppData (%ProgramData%) - and none for AppData or Local AppData, as Microsoft Learn says. The other names rubrapack uses (ProgramFilesX86, ProgramFilesCommonX64/X86, SystemX86, Windows) are those Microsoft Learn lists. [spec] [observed]

15.7 Extensions

What rubrapack writes into an application's <Extensions> (after uap:VisualElements), and only this: a source feature with no row here is an error, and one whose Windows build is above the package's MinVersion asks for min-version to be raised. [spec] Microsoft Learn, "Integrate your desktop app with Windows using packaging extensions" and the element pages. The last column is what was checked by installing a package on Windows 11 (26100). [observed]

Source	Element (category)	Namespace	Min build	Capability	Checked on Windows 11
[assoc]	uap:Extension windows. fileTypeAssociation > uap3:FileTypeAssociation (Name, Parameters) > uap: DisplayName, uap: SupportedFileTypes > uap: FileType	uap, uap3	14393	runFullTrust	opening a .rpx/.rpy file starts the program with the file
[protocol]	uap3:Extension windows. protocol > uap3:Protocol (Name, Parameters)	uap3	14393	runFullTrust	starting a scheme: URI starts the program with it
[msix-extension] alias	uap3:Extension windows. AppExecutionAlias (Executable, EntryPoint) > uap3:AppExecutionAlias > desktop:ExecutionAlias (Alias)	uap3, desktop	14393	runFullTrust	the alias appears in % LOCALAPPDATA%\ Microsoft\WindowsApps and starts the program
[msix-extension] startup task	desktop:Extension windows.startupTask (Executable, EntryPoint) > desktop:StartupTask (TaskId, Enabled, DisplayName)	desktop	14393	runFullTrust	registered after the first start
[shortcut] Desktop	desktop7:Extension windows.shortcut > desktop7:Shortcut (File \$(Desktop)\<name>.lnk, Icon, Arguments, Description)	desktop7	19645	runFullTrust	the shortcut is on the user's desktop and starts the program
[shortcut] Programs StartMenu	one: the application's own Start entry	-	-	-	the Start menu lists the application

Source	Element (category)	Namespace	Min build	Capability	Checked on Windows 11
[font]	uap4:Extension windows. sharedFonts > uap4: SharedFonts > uap4:Font (File Fonts\<name>), in the first application	uap4	15063	-	other programs see the font while the package is installed, and not after

- The namespaces are declared on Package, and made ignorable, only when used, so a package without extensions keeps the manifest described above.
- Parameters, Arguments: literal text (MSI's [...] is refused); Windows puts the file or URI where %1 is.
- FileTypeAssociation Name: the prog-id in lower case; [assoc] tables that share a prog-id become one association with several FileTypes.
- Everything goes into the application whose executable the source names as target; a target that is not an application's executable is an error.

15.8 Bundles (.msixbundle)

A ZIP archive laid out like a package (ZIP64 entries with data descriptors), as Windows' bundle writer (IAppxBundleWriter) makes it: [observed]

- The packages first, stored (method 0) under their file names, in the order they were added; then AppxMetadata/AppxBundleManifest.xml, AppxBlockMap.xml and [Content_Types].xml, deflated.
- The block map lists the bundle manifest only (AppxMetadata\AppxBundleManifest.xml); the packages carry their own block maps.
- [Content_Types].xml: Default msix = application/vnd.ms-appx, Default xml = application/vnd.ms-appx.bundlemanifest+xml, and an Override for /AppxBlockMap.xml.
- The bundle manifest, CRLF line ends, tab indents, no line end after the last tag:

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<Bundle xmlns="http://schemas.microsoft.com/appx/2013/bundle" SchemaVersion="5.0"
xmlns:b4="http://schemas.microsoft.com/appx/2018/bundle" xmlns:b5="http://schemas.microsoft.com/appx/2019/bundle" IgnorableNamespaces="b4 b5">
  <Identity Name="Example.App" Publisher="CN=Example" Version="1.0.0.0"/>
  <Packages>
    <Package Type="application" Version="1.0.0.0" Architecture="x64"
FileName="Example.App_1.0.0.0_x64.msix" Offset="66" Size="61340">
      <Resources>
        <Resource Language="en-US"/>
      </Resources>
      <b4:Dependencies>
        <b4:TargetDeviceFamily Name="Windows.Desktop" MinVersion="10.0.17763.0"
MaxVersionTested="10.0.26100.0"/>
      </b4:Dependencies>
    </Package>
  </Packages>
</Bundle>
```

- Offset is where the package's bytes start in the bundle (after its local header), Size their length. Resources and Dependencies repeat the package manifest's Resource and TargetDeviceFamily elements.
- The bundle's Version is the writer's argument (a 64-bit number, 16 bits per part); rubrapack gives the packages' version. Every package must have the bundle's Name and Publisher, one version, and an architecture of its own.
- rubrapack's bundle of the same packages has the same entries at the same offsets and the same manifest, block map and content types, byte for byte; only the ZIP dates differ (1980

in rubrapack's). Windows installs from it the package for its own architecture: x64 on an x64 machine even when x86 and arm64 are there, x86 from a bundle of x86 alone. [observed]

15.9 Signatures (AppxSignature.p7x)

Worked out against Windows' signer (mssign32!SignerSignEx2 with APPX_SIP_CLIENT_DATA; the PowerShell cmdlet cannot sign a package) and checked by installing rubrapack's signed packages. [observed]

- Publisher in the manifest must be the signing certificate's subject written the way Windows displays it: the RDNs from last to first, A=value joined by , - a certificate made with /CN=Example/O=Example Ltd needs O=Example Ltd, CN=Example. Otherwise signing fails with 0x8007000B.
- Signing rewrites the archive the way the signer does: the payload's local records, the manifest and the block map stay as they were; [Content_Types].xml gets <Override PartName="/AppxSignature.p7x" ContentType="application/vnd.ms-appx.signature"/>; the signature is added last, deflated, with its sizes in the local header (version needed 2.0, no data descriptor); the central directory is written the ZIP32 way (no ZIP64 extra fields) where sizes and offsets allow, and the end record's disk numbers are 0.
- AppxSignature.p7x is PKCX followed by a CMS SignedData as in Authenticode signatures, with these differences: data is SEQUENCE { SpcSipInfo (1.3.6.1.4.1.311.2.1.30), SEQUENCE { INTEGER 0x01010000, OCTET STRING <SIP GUID>, INTEGER 0, INTEGER 0, INTEGER 0, INTEGER 0, INTEGER 0 } } with the package SIP GUID bytes 4B DF C5 0A 07 CE E2 4D B7 6E 23 C8 39 A0 9F D1 or the bundle SIP GUID bytes B3 58 5F 0F DE AA 9A 4B A4 34 95 74 2D 92 EC EB; the signed attributes are contentType and messageDigest only.
- The DigestInfo's digest is not one hash but APPX followed by records of a 4-byte tag and a SHA-256:
 - AXPC: the archive from its start to the signature entry's local header;
 - AXCD: the central directory without the signature entry, then the ZIP64 end record, its locator and the end record, all as if the signature entry did not exist (the central directory starting where the signature's local header is). Windows reads the signature entry's place from its ZIP32 fields: a signature entry described the ZIP64 way (0xFFFFFFFF there) is a HashMismatch - so is the package whose central directory is rewritten that way after signing;
 - AXCT: [Content_Types].xml (the plain bytes); AXBM: AppxBlockMap.xml;
 - AXCI: AppxMetadata/CodeIntegrity.cat, only when the package has one.
- Windows' signer also adds AppxMetadata/CodeIntegrity.cat - a catalog of the package's program files, signed by the same key - when the package holds any. It is optional: a package signed without it installs and runs. rubrapack does not write one; it checks AXCI when it is there.
- A bundle: every package inside is signed first (each with its own AppxSignature.p7x), then the bundle is written around them and signed with the bundle SIP GUID, without AXCI.
- An unsigned package that needs -AllowUnsigned and the publisher OID (below) is not signed: signing refuses a publisher with that OID.

15.10 Installing an unsigned package

- Publisher must end in 01D.2.25.311729368913984317654407730594956997722=1; then Add-AppxPackage -AllowUnsigned installs it on Windows 11. [spec]
- A package with an executable needs an elevated process (otherwise 0x80073D2B: an unsigned package cannot hold an executable activation). Developer mode is not needed. [observed]
- Add-AppxPackage from a network logon (an SSH session) fails at "PLM initialization" with 0x80070005; it works in an interactive session. [observed]
- A package with file types leaves, after removal, an empty OpenWithProgids key under HKCU\Software\Classes\.<ext> - Windows' own doing, not the package's. [observed]

16 Registry hive files (REGF)

In this chapter

16.1	Layout	71
16.2	The base block	71
16.3	Keys (nk)	72
16.4	Subkey lists (lh, ri)	72
16.5	Values (vk) and value lists	72
16.6	The security cell (sk)	72
16.7	Reproducible hives	72

The file format of Registry.dat and User.dat in an MSIX package. Microsoft does not publish it; every fact here is **[observed]**: hives written by Windows' Offline Registry Library (offreg.dll, 10.0.26100, the writer Windows' packaging tools use - its mark is in the base block) for chosen key/value sets, and rubrapack's hives for the same sets read back on Windows with OROpenHive and RegLoadAppKey into exactly the keys and values Windows enumerates in its own. Only what a freshly written hive needs is described: no transaction logs, no volatile keys, no free-cell reuse.

16.1 Layout

- A 4096-byte base block, then hive bins. Offsets in cells ("cell offsets") count from the end of the base block (file offset - 4096); "none" is 0xFFFFFFFF.
- A bin: hbin, u32 its own cell offset, u32 its size (4096 or a multiple), 8 bytes zero, u64 time (0), u32 spare (0); 32 bytes in all. Cells follow; a cell that does not fit in the current bin starts a new bin sized to hold it; the rest of a bin is one free cell.
- A cell: i32 size (negative while in use, a multiple of 8, the size field included), then its content. A cell never crosses its bin's end.

16.2 The base block

Offset	Field	Value
0	signature	regf
4, 8	sequence numbers	1, 1 (unequal = not closed cleanly; readers refuse or replay a log)
12	u64 time	0 is accepted
20, 24	version	1, 5
28, 32	type, format	0, 1
36	root key cell offset	
40	size of all bins	
44	clustering	1
0xB0	offreg's mark	0fRg, then u32 1
508	checksum	XOR of the 127 dwords before it; 0 is written as 1, 0xFFFFFFFF as 0xFFFFFFFFE

Everything else is zero.

16.3 Keys (nk)

nk, u16 flags, u64 time, u32 access bits (0), u32 parent (none for the root), u32 subkey count, u32 volatile subkey count (0), u32 subkey list, u32 volatile list (none), u32 value count, u32 value list, u32 security cell, u32 class (none), u32 longest subkey name, u32 longest class (0), u32 longest value name, u32 largest value data, u32 work var (0), u16 name length, u16 class length (0), then the name.

- Flags: 0x20 when the name is stored as Latin-1 ("compressed": every UTF-16 unit below 256), else the name is UTF-16LE. The root key of an offreg hive is named ROOT and has flags 0x20 only.
- The longest names are counted in UTF-16 bytes, whatever the stored form.
- Names compare without case: each UTF-16 unit mapped by Unicode simple upper case.

16.4 Subkey lists (lh, ri)

- lh, u16 count, then per subkey: u32 key cell, u32 hash. Sorted by upper-cased name.
- Hash: $h = 0$; for each UTF-16 unit of the name, $h = h * 37 + \text{upper}(\text{unit})$ (32-bit wrap).
- More than 507 subkeys: an ri (u16 count, u32 per lh list) of lh lists of up to 507.

16.5 Values (vk) and value lists

- A value list is a plain cell of u32 value cell offsets, in the order the values were set.
- vk, u16 name length, u32 data size, u32 data offset, u32 type, u16 flags, u16 spare, then the name. Flag 1 = Latin-1 name; the unnamed default value has an empty name and flags 0.
- Data of up to 4 bytes (an empty value too) is kept in the data offset field, and the size's top bit is set.
- Up to 16344 bytes: one data cell. From 16345 bytes: a db cell (db, u16 segment count, u32 segment list), a list cell of u32 segment cells, and segments of 16344 bytes each - every segment cell allocated at full size, the last one too: Windows stops reading a value whose last segment cell is shorter.

16.6 The security cell (sk)

One sk cell shared by every key: sk, u16 0, u32 next and u32 previous (both itself), u32 reference count (the number of keys), u32 descriptor length, then a self-relative security descriptor. offreg's: owner and group Administrators; SYSTEM and Administrators full access, Everyone and RESTRICTED read, all container-inherit.

16.7 Reproducible hives

With every time field 0 and cells laid out depth first (a key; after the root key the security cell; the key's subkey lists, its value list, each value followed by its data; then its subkeys in sorted order), the same keys and values give the same bytes on any host. Windows reads such a hive without complaint.